Latest updates: https://dl.acm.org/doi/10.1145/3319535.3339820

RESEARCH-ARTICLE

LegoSNARK: Modular Design and Composition of Succinct Zero-Knowledge Proofs

MATTEO CAMPANELLI, IMDEA Software Institute, Pozuelo de Alarcon, Madrid, Spain

DARIO FIORE, IMDEA Software Institute, Pozuelo de Alarcon, Madrid, Spain

ANAÏS QUEROL, Technical University of Madrid, Madrid, Madrid, Spain

Open Access Support provided by:

Technical University of Madrid

IMDEA Software Institute

PDF Download
3319535.3339820.pdf
17 January 2026
Total Citations: 130
Total Downloads: 1094

Published: 06 November 2019

Citation in BibTeX format

CCS '19: 2019 ACM SIGSAC Conference on Computer and Communications Security
November 11 - 15, 2019
London, United Kingdom

Conference Sponsors:
SIGSAC

CCS '19: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (November 2019)
https://doi.org/10.1145/3319535.3339820
ISBN: 9781450367479

LegoSNARK: Modular Design and Composition of Succinct Zero-Knowledge Proofs

Matteo Campanelli
IMDEA Software Institute
Madrid, Spain
matteo.campanelli@imdea.org

Dario Fiore
IMDEA Software Institute
Madrid, Spain
dario.fiore@imdea.org

Anaïs Querol
IMDEA Software Institute
Universidad Politécnica de Madrid
Madrid, Spain
anais.querol@imdea.org

ABSTRACT

We study the problem of building non-interactive proof systems modularly by linking small specialized "gadget" SNARKs in a lightweight manner. Our motivation is both theoretical and practical. On the theoretical side, modular SNARK designs would be flexible and reusable. In practice, specialized SNARKs have the potential to be more efficient than general-purpose schemes, on which most existing works have focused. If a computation naturally presents different "components" (e.g. one arithmetic circuit and one boolean circuit), a general-purpose scheme would homogenize them to a single representation with a subsequent cost in performance. Through a modular approach one could instead exploit the nuances of a computation and choose the best gadget for each component.

Our contribution is LegoSNARK, a "toolbox" (or framework) for commit-and-prove zkSNARKs (CP-SNARKs) that includes:
1) General composition tools: build new CP-SNARKs from proof gadgets for basic relations *simply*.
2) A "lifting" tool: a compiler to add commit-and-prove capabilities to a broad class of existing zkSNARKs *efficiently*. This makes them interoperable (linkable) within the same computation. For example, one QAP-based scheme can be used prove one component; another GKR-based scheme can be used to prove another.
3) A collection of succinct proof gadgets for a variety of relations.

Additionally, through our framework and gadgets, we are able to obtain new succinct proof systems. Notably:
– LegoGro16, a commit-and-prove version of Groth16 zkSNARK, that operates over data committed with a classical Pedersen vector commitment, and that achieves a 5000× speedup in proving time.
– LegoUAC, a pairing-based SNARK for arithmetic circuits that has a universal, circuit-independent, CRS, and proving time linear in the number of circuit gates (vs. the recent scheme of Groth et al. (CRYPTO'18) with quadratic CRS and quasilinear proving time).
– LegoMM, a CP-SNARK for matrix multiplication that achieves optimal proving complexity.

CCS CONCEPTS

• **Security and privacy → Public key (asymmetric) techniques**.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CCS '19, November 11–15, 2019, London, United Kingdom
© 2019 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-6747-9/19/11…$15.00
https://doi.org/10.1145/3319535.3339820

KEYWORDS

zero-knowledge, zkSNARKs, verifiable computation

ACM Reference Format:
Matteo Campanelli, Dario Fiore, and Anaïs Querol. 2019. LegoSNARK: Modular Design and Composition of Succinct Zero-Knowledge Proofs. In *2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19), November 11–15, 2019, London, United Kingdom.* ACM, New York, NY, USA, 18 pages. https://doi.org/10.1145/3319535.3339820

1 INTRODUCTION

Zero-knowledge proofs (ZKPs), introduced by Goldwasser, Micali and Rackoff [36], let a prover convince a verifier of a statement without revealing more information than its validity. This power of ZKPs—simultaneously providing *integrity* (the prover cannot cheat) and *privacy* (the verifier does not learn any of the prover's secrets)—has found countless applications, including multiparty computation [34], signature schemes [59], public-key encryption [54], and, more recently, blockchain systems [6, 10].

Some zero-knowledge proof systems—called *succinct* or simply *zkSNARKs*, zero-knowledge Succinct Non-interactive Argument of Knowledge—have short and efficiently verifiable proofs [15, 32, 53]. Succinctness is desirable in general but is especially critical in applications where verifiers would not invest significant computational resources (e.g. if they are unwilling to do it for reasons of scalability and cost, or if they are computationally weak).

Motivation. The last years have seen remarkable progress in the construction of zkSNARKs. Different lines of work (cf. Section 1.2 for a detailed review) have built a variety of schemes that are highly expressive, supporting general computations in the class NP. The general-purpose nature of these schemes makes them very attractive to practitioners. At the same time, this high expressivity comes at a cost in terms of performance. To achieve generality, these constructions abstract specific features of computation by assuming one *single unifying representation* (e.g., boolean or arithmetic circuits, state-machine transitions, RAM computations), and this abstraction is often a source of overhead, for two main reasons.

First, *general-purpose zk-SNARKs may miss opportunities for significant optimizations* by not exploiting the nuances of a computation. In contrast, specialized solutions can gain efficiency by exploiting specific structural properties. For example, recent works [25, 65] show how to highly optimize the GKR protocol[35] for the case of parallel computations. A further example is the specialized protocol for the multiplication of $n \times n$ matrices we propose in Section 5.5. Here, our prover runs in $O(n^2)$ time as opposed to a circuit-based approach running in at least $O(n^3)$ time.

Second, *computation tends to be heterogeneous*, often consisting of several subroutines of different nature, e.g. both arithmetic and boolean components. If we design SNARKs assuming one single general representation then we will not be able to provide the best match for all the different subroutines. In this context specialized protocols are clearly not an answer either as they fail whenever faced with a non homogeneous computation. As a concrete example, the GKR-like protocols mentioned above are highly efficient when executed on parallel computations, but they fail to be succinct if a computation *also* includes heavily sequential subroutines (e.g. iterated block ciphers).

A Modular Approach for zk-SNARKs. In this paper we study an alternative approach to the design of zkSNARKs that would gain the advantages of specialized proof systems without inheriting their shortcomings when applied to heterogeneous computations. With this goal in mind we propose to build zkSNARKs by proceeding in a modular "bottom-up" fashion. Most current works use a "top-down" approach: they build general-purpose schemes adopting one single representation that *must* be shared across all the different subroutines in the program. On the other hand, in this work we consider designing a "global" SNARK for a computation C through a (lightweight) linking of "smaller" specialized SNARKs for the different subroutines composing C. We call these interlinked specialized SNARKs *proof gadgets*, as they act as basic building blocks that one can compose and reuse according to the situation. This approach inherits all the benefits of modularity, e.g. reducing complexity (one can focus on specific, simpler problems), increased flexibility and costs reduction (proof gadgets can be reused, replaced, etc.).

Modularity from Commit-and-Prove SNARKs. To realize this modular approach we rely on the well known *commit-and-prove* (CP) methodology [22, 44]. With a CP scheme one can prove statements of the form "$c_{\mathsf{ck}}(x)$ contains x such that $R(x, w)$" where $c_{\mathsf{ck}}(x)$ is a commitment. To see how the CP capability can be used for modular composition consider the following example of sequential composition in which one wants to prove that $\exists w : z = h(x; w)$, where $h(x; w) := g(f(x; w); w)$. Such a proof can be built by combining two CP systems Π_f and Π_g for its two building blocks, i.e., respectively f and g: the prover creates a commitment $c_{\mathsf{ck}}(y)$ of y, and then uses Π_f (resp. Π_g) to prove that "$c_{\mathsf{ck}}(y)$ contains $y = f(x; w)$ (resp. contains y such that $z = g(y; w)$)".

Challenges of the CP modular composition. The composition idea sketched above implicitly assumes that Π_f and Π_g work on the same commitment $c_{\mathsf{ck}}(y)$. Namely, *in order to be composed, different CP schemes must be compatible with the same commitment scheme* (and commitment key). Essentially we need a *sort of universal commitment scheme* that is as decoupled as possible from the specific argument systems that will operate on it.

We argue that achieving such universality with state-of-the-art zkSNARKs entails major challenges:

(a) Most of the popular zkSNARKs, e.g., [39, 55], are not explicitly commit-and-prove. This limitation can be overcome using a (somewhat folklore) approach in which the SNARK Π additionally proves the correct opening of the commitment, i.e., $R(x, w) \wedge$ "$c_{\mathsf{ck}}(x)$ opens to x". This approach has two main drawbacks: *(i)* Π must be expressive enough to include the commitment verification in its language, but in our vision Π is a SNARK for a specialized task and may not have this capability; *(ii)* even if Π were expressive enough (e.g., supports arbitrary circuits), encoding commitment verification incurs significant overheads.[1]

(b) Some existing SNARKs have commit-and-prove capabilities [26, 38, 49, 62]. Yet, each of these schemes uses its own specific commitment scheme. In some cases [26] the commitment keys are *relation-dependent*, which means commitments cannot be generated before fixing one or multiple relations.[2] In the other cases, despite being relation-independent, commitment keys have a very specific structure that may not fit other proof systems. In summary, a main limitation of existing commit-and-prove SNARKs is their incompatibility, between them and with other potentially more efficient candidates to be developed.

1.1 Our Results

LegoSNARK Framework. We present LegoSNARK, a framework for commit-and-prove zkSNARKs (CP-SNARKs) that includes:

- *Definitions* that formalize CP-SNARKs and their variants.
- *Composition recipes* that show how to use different CP-SNARKs in a generic and secure way for handling conjunction, disjunction and sequential composition of relations. This composition result enables the use of modularity in designing CP-SNARKs for complex relations out of schemes for simpler relations.
- *A generic construction* to efficiently turn a broad class of zk-SNARKs into CP-SNARKs that can be composed together. This class includes several existing schemes such as ones based on quadratic arithmetic programs [26, 39, 55], or zk-vSQL [67, 68]. For this transformation we only need a "minimal" CP-SNARK, $\mathsf{CP}_{\mathsf{link}}$, for proving that two commitments (under different schemes) open to the same value.

LegoSNARK Gadgets. We populate our framework by constructing new CP-SNARKs for several basic relations, such as:

- $\mathsf{CP}_{\mathsf{link}}$ for proving that two *different* Pedersen-like commitments open to the same vector.[3] Plugging $\mathsf{CP}_{\mathsf{link}}$ in our generic construction solves the challenges (a) and (b) mentioned above and gives us interoperable versions of several existing schemes.
- $\mathsf{CP}_{\mathsf{lin}}$ for proving that a linear relation $\boldsymbol{F} \cdot \boldsymbol{u} = \boldsymbol{x}$ holds for a committed vector $\boldsymbol{u}$, a public matrix $\boldsymbol{F}$ and public vector $\boldsymbol{x}$.
- $\mathsf{CP}_{\mathsf{had}}$ for proving that a vector $\boldsymbol{u}_0$ is the Hadamard product of $\boldsymbol{u}_1$ and $\boldsymbol{u}_2$, when all the three vectors are committed.
- $\mathsf{CP}_{\mathsf{sfprm}}$ for proving a self-permutation, i.e., that $y_i = y_{\phi(i)}$ for a public permutation ϕ and a committed vector $\boldsymbol{y}$.
- $\mathsf{CP}_{\mathsf{mm}}$ for proving that matrix $\boldsymbol{X}$ is the product of committed matrices $\boldsymbol{A}$ and $\boldsymbol{B}$.

All the aforementioned schemes have succinct proofs and work for Pedersen-like commitments in bilinear groups. This means that by using our generic construction with $\mathsf{CP}_{\mathsf{link}}$ they can be turned to support the same commitment and then be composed.

[1] For example, we experimentally found that, when handling a Pedersen commitment to a vector of length 2048 with [39], the proving overhead is 428 secs (7 minutes).
[2] This could be mitigated by using universal circuits, paying a (multiplicative) logarithmic overhead in parameters size and prover complexity.
[3] By "Pedersen-like" we mean schemes where the verification algorithm is the same as in Pedersen scheme [56] for vectors (but the bases can have a different distribution).

LegoSNARK Applications and Evaluation. Using our initial set of specialized proof gadgets, our next step is to combine them in order to build new succinct proof systems for different use cases, mentioned below. Our results offer various improvements over the state of the art. We have also implemented some of our solutions to test their concrete performance.

1) Efficient Commit-Ahead-of-Time. Through our generic construction instantiated with $\mathsf{CP}_{\mathsf{link}}$ we also obtained commit-and-prove versions of popular efficient zkSNARKs, such as Groth's [39], that can prove statements about data committed using the Pedersen scheme for vectors [56], in which bases are random group elements that can be generated without trusted setup. Such commit-and-prove schemes are useful in applications where one needs to commit *before* the SNARK keys for a relation are created, e.g., to post commitments on a blockchain so that one can later prove statements about the committed data. By applying our solution to [39] we obtain a scheme that is 5000× faster than Groth16, where the commitment is encoded in the circuit.

2) CP-SNARKs for Parallel Computation. Consider the problem of proving (in zero-knowledge) correctness of a computation that consists of the same subcircuit executed in parallel. The recent Hyrax system [66] is suitably designed for and shows good performances on this type of circuit. It requires, however, an additional verification cost whenever the repeated subcircuits share (non-deterministic) inputs, which is common. The verifier thus pays an additional factor linear in the total width of the circuit. Using our LegoSNARK framework we show how to build a new CP-SNARK based on Hyrax that avoids this problem. The idea is that parallel computation on joint inputs can be expressed as the combination of a fully parallel computation (after inputs were appropriately duplicated) and a permutation check to ensure that inputs have been duplicated correctly. We build this by combining our $\mathsf{CP}_{\mathsf{lin}}$ gadget with a version of Hyrax modified to work with the polynomial commitment of zk-vSQL [68].

3) CP-SNARKs for Arithmetic Circuits. We give two main constructions of CP-SNARKs for arithmetic circuit (AC) satisfiability. Table 1 summarizes a theoretical comparison with other schemes in the literature (selected among the ones with similar succinctness).

Our first scheme, LegoAC, relies on an encoding of AC based on Hadamard products and linear constraints from [16] and can be built from $\mathsf{CP}_{\mathsf{lin}}$ and $\mathsf{CP}_{\mathsf{had}}$ gadgets. We evaluate two instantiations:

- LegoAC1—from our $\mathsf{CP}_{\mathsf{lin}}$ and a $\mathsf{CP}_{\mathsf{had}}$ from [49]— is secure in the generic group model (GGM), enjoys constant-size proofs, and has a $\log n$ factor in proving time (similar to [39, 55]);
- LegoAC2—from our $\mathsf{CP}_{\mathsf{lin}}$ and $\mathsf{CP}_{\mathsf{had}}$ gadgets—is secure in the GGM+ROM, it has $\log n$-size proofs but only *linear* proving time.

The second CP-SNARK, LegoUAC, builds on an encoding of AC based on Hadamard products, additions and permutation from [17, 37] and can be built from our $\mathsf{CP}_{\mathsf{had}}$ and $\mathsf{CP}_{\mathsf{sfprm}}$ gadgets.[4] The main novelty of LegoUAC is to admit a *universal, circuit-independent* CRS, in the "specialization" model of [40] where the universal CRS can be specialized to a circuit C with a deterministic algorithm. LegoUAC's CRS has $O(N)$ size where N is an upper bound on the number of gates of the circuits; in contrast, the CRS has quadratic size in the recent scheme in [40] Our LegoUAC also improves on approaches applying an efficient system, say [39], on a universal circuit [41, 61], which incur at least a logarithmic multiplicative factor in circuit size.

[4]Additions are handled for free if the commitment is linearly homomorphic.

Scheme	Uni	KG time	Prove time	Ver. time	$\lvert\mathsf{crs}\rvert$	$\lvert\pi\rvert$
[39, 55]	–	$n+m$	$m+n\log n$	$\lvert x\rvert$	$n+m$	$O(1)$
LegoAC1	–	$n+m$	$n\log n$	$\lvert x\rvert$	n	$O(1)$
LegoAC2	–	$n+m$	n	$\lvert x\rvert+\log n$	n	$\log n$
[40]	✓	n^2	$m+n\log n$	$\lvert x\rvert$	n^2	$O(1)$
LegoUAC	✓	N^*	N	$\lvert x\rvert+\log N$	N^*	$\log N$

Table 1: Comparing pairing-based zkSNARKs for arithmetic circuits with m wires and N gates, of which n are multiplication gates, n_a (resp. n_c) are addition (resp. multiplication-by-constant) gates, and $N^* = \max(n, n_a, n_c)$. Numbers in the table are in $O(\cdot)$ notation.

1.2 Related Work

The idea of combining two different NIZKs to improve efficiency when handling heterogeneous computations has been considered by Chase et al. [24] and more recently by Agrawal et al. [5]. In [5], they propose combining the Pinocchio scheme [55] with Sigma-protocol-based NIZKs and show an efficient construction for computations that combines algebraic relations in a cryptographic group and arbitrary computation. Their approach reveals beneficial and improves performances. The solution in [5] is tailored to two specific proof systems and their combination methodology does not always preserve succinctness. In contrast, our techniques are *general*, apply to a variety of existing proof systems and preserve succinctness (they compose succinct schemes into succinct schemes).

Succinct ZK Proofs. In the past years several research lines have built a variety of zk-SNARKs for general NP statements. Here we provide an overview of each line, especially focusing on their differences in performance.

A major research line is the one based on the seminal paper of Gennaro et al. [31] who proposed a pairing-based SNARK based on the NP-complete language of quadratic span/arithmetic programs. This approach improves on previous approaches by Ishai et al. [42], Groth [38] and Lipmaa [48], and is the basis of several works such as [8, 11, 13, 19, 26, 29, 39, 40, 47, 55, 64]. The zkSNARKs in this family enjoy constant-size proofs and fast verification, the latter depending only linearly on the statement size; on the downside, they feature large overheads at proving time, costly (although amortizable) preprocessing and security properties based on non-standard non-falsifiable assumptions.

A second research line builds on the MPC-in-the-head approach of Ishai et al. [43] to construct a ZK argument from an MPC protocol. The first scheme that refined and experimented this approach is ZKBoo [33], then improved in [23]; a more recent work in this line is Ligero [7]. These schemes do not need trusted setup and show excellent proving performances on Boolean circuits, since they rely only on symmetric-key cryptographic primitives. On the downside their proofs are not fully succinct, being linear in the circuit size $|C|$ in [33], and $\tilde{O}(\sqrt{|C|})$ in [7].

The works [66–68] stem from the interactive proof techniques for low-depth circuits pioneered in Goldwasser et al. [35] and later refined in [25, 60, 63]. The resulting succinct ZK arguments are made non-interactive in the random oracle model. These schemes

offer good proving performance and use asymptotically fewer cryptographic operations than those from the MPC-in-the-head family; they can be instantiated without [66] (or with a circuit-independent [68]) trusted setup. On the other hand their proof size and verification time depend on the structure of the circuit at hand, notably on the depth and in some cases on the width.

Building on the work of Groth [37], two recent proposals [16, 20] give ZK arguments for arithmetic circuit satisfiability that can be instantiated without trusted setup. The first scheme of Bootle et al. [16] has proofs of size $O(\sqrt{M})$ where M is the number of multiplication gates in the circuit, while their second scheme (improved in [20]) has proofs of size $O(\log M)$ but has a linear time verifier.

Compared to the results from the latter three research lines we described, our instantiations have the disadvantage of needing a trusted setup[5], although in some cases this is universal and thus reusable. In terms of performances, however, our results are more succinct, both in terms of proof size and verifier time.

A recent line of work [9] builds on the seminal works of Kilian [45] and Micali [52], and generalizations of PCPs (IOPs) [12, 57] in order to construct systems (dubbed zkSTARKs) that are general-purpose (capturing very general computations that can be expressed as state-machine transitions), do not require trusted setup and offer good timings for prover and verifier. On the downside, the memory costs for the prover are still high and their security relies on a non-standard conjecture about Reed-Solomon codes.

1.3 Roadmap

The paper is organized as follows. Section 2 introduces notation and preliminar definitions. Section 3 provides the basis for building our framework: composing CP-SNARKs, the notion of cc-SNARKs and our compiler to import existing schemes in the framework. Sections 4 and 5 present constructions both for Pedersen-like commitments and polynomial commitments. Section 6 explains how to apply LegoSNARK to build schemes for arithmetic circuits. Section 7 gives experimental details of our library. We conclude in Section 8.

This work is a short form of our full version available at [21]. Due to space constraints, several results only appear in the long version. Namely: security proofs, formal definitions, more schemes and constructions, and further details. We refer the reader to this version now to avoid redundancy along the text.

2 PRELIMINARIES

We use $\lambda \in \mathbb{N}$ to denote the security parameter, and 1^λ to denote its unary representation. Throughout the paper we assume that all the algorithms of the cryptographic schemes take as input 1^λ, and thus we omit it from the list of inputs. For a distribution D, we denote by $x \leftarrow D$ the fact that x is being sampled according to D. We remind the reader that an ensemble $\mathcal{X} = \{X_\lambda\}_{\lambda\in\mathbb{N}}$ is a family of probability distributions over a family of domains $\mathcal{D} = \{D_\lambda\}_{\lambda\in\mathbb{N}}$. We say two ensembles $\mathcal{D} = \{D_\lambda\}_{\lambda\in\mathbb{N}}$ and $\mathcal{D}' = \{D'_\lambda\}_{\lambda\in\mathbb{N}}$ are statistically indistinguishable (denoted by $\mathcal{D} \approx_s \mathcal{D}'$) if $\frac{1}{2}\sum_x |D_\lambda(x) - D'_\lambda(x)| < \mathsf{negl}(\lambda)$. If $\mathcal{A} = \{\mathcal{A}_\lambda\}$ is a (possibly non-uniform) family of circuits and $\mathcal{D} = \{D_\lambda\}_{\lambda\in\mathbb{N}}$ is an ensemble, then we denote by $\mathcal{A}(\mathcal{D})$ the ensemble of the outputs of $\mathcal{A}_\lambda(x)$ when $x \leftarrow D_\lambda$. We say two ensembles $\mathcal{D} = \{D_\lambda\}_{\lambda\in\mathbb{N}}$ and $\mathcal{D}' = \{D'_\lambda\}_{\lambda\in\mathbb{N}}$ are computationally indistinguishable (denoted by $\mathcal{D} \approx_c \mathcal{D}'$) if for every non-uniform polynomial time distinguisher $\mathcal{A}$ we have $\mathcal{A}(\mathcal{D}) \approx_s \mathcal{A}(\mathcal{D}')$.

We denote by $[n]$ the set of integers $\{1, \ldots, n\}$ and by $[: n]$ the set $\{0, 1, \ldots, n-1\}$. By $(u_j)_{j\in[\ell]}$ we denote the tuple $(u_1, \ldots, u_\ell)$.

Relations. Let $\{\mathcal{R}_\lambda\}_{\lambda\in\mathbb{N}}$ be a family of polynomial-time decidable relations R on pairs (x, w) where $x \in \mathcal{D}_x$ is called the *statement* or *input*, and $w \in \mathcal{D}_w$ the *witness*. We write $R(x, w) = 1$ to denote that R holds on (x, w), else we write $R(x, w) = 0$. When discussing schemes that prove statements on committed values we assume that $\mathcal{D}_w$ can be split in two subdomains $\mathcal{D}_u \times \mathcal{D}_\omega$. Finally we sometimes use an even finer grained specification of $\mathcal{D}_u$ assuming we can split it over ℓ arbitrary domains $(\mathcal{D}_1 \times \cdots \times \mathcal{D}_\ell)$ for some arity ℓ. In our security definitions we assume relations to be generated by a *relation generator* $\mathcal{RG}(1^\lambda)$ that, on input the security parameter 1^λ, outputs R together with some side information, an auxiliary input aux_R, that is given to the adversary. We define $\mathcal{RG}_\lambda$ as the set of all relations that can be returned by $\mathcal{RG}(1^\lambda)$.

Commitments. We recall non-interactive commitment schemes.

Definition 2.1. A commitment scheme is a tuple of algorithms $\mathsf{Com} = (\mathsf{Setup}, \mathsf{Commit}, \mathsf{VerCommit})$ that work as follows and satisfy the notions of *correctness*, *binding* and *hiding* defined below.

- $\mathsf{Setup}(1^\lambda) \rightarrow \mathsf{ck}$ takes the security parameter and outputs a commitment key ck. This key includes descriptions of the input space $\mathcal{D}$, commitment space C and opening space O.
- $\mathsf{Commit}(\mathsf{ck}, u) \rightarrow (c, o)$ takes the commitment key ck and a value $u \in \mathcal{D}$, and outputs a commitment c and an opening o.
- $\mathsf{VerCommit}(\mathsf{ck}, c, u, o) \rightarrow b$ takes as input a commitment c, a value u and an opening o, and accepts ($b = 1$) or rejects ($b = 0$).

Correctness. For all $\lambda \in \mathbb{N}$ and any input $u \in \mathcal{D}$ we have:

$$\Pr\left[\begin{array}{l}\mathsf{ck} \leftarrow \mathsf{Setup}(1^\lambda)\\ (c, o) \leftarrow \mathsf{Commit}(\mathsf{ck}, u)\end{array} : \mathsf{VerCommit}(\mathsf{ck}, c, u, o) = 1\right] = 1$$

Binding. For every polynomial-time adversary $\mathcal{A}$

$$\Pr\left[\begin{array}{l}\mathsf{ck} \leftarrow \mathsf{Setup}(1^\lambda)\\ (c, u, o, u', o') \leftarrow \mathcal{A}(\mathsf{ck})\end{array} : \begin{array}{l}\mathsf{VerCommit}(\mathsf{ck}, c, u', o')\\ \wedge\ \mathsf{VerCommit}(\mathsf{ck}, c, u, o)\\ \wedge\ u \neq u'\end{array}\right] = \mathsf{negl}$$

Hiding. For $\mathsf{ck} \leftarrow \mathsf{Setup}(1^\lambda)$ and every values $u, u' \in \mathcal{D}$, we require $\mathsf{Commit}(\mathsf{ck}, u) \approx \mathsf{Commit}(\mathsf{ck}, u')$.

2.1 Zero-Knowledge SNARKs

We recall the definition of (pre-processing) zkSNARKs [14, 15].

Definition 2.2 (SNARK). A succinct non-interactive argument of knowledge for $\{\mathcal{R}_\lambda\}_{\lambda\in\mathbb{N}}$ is a tuple of three algorithms $\Pi = (\mathsf{KeyGen}, \mathsf{Prove}, \mathsf{VerProof})$ that work as follows and satisfy the notions of *completeness*, *succinctness* and *knowledge soundness* defined below. If Π also satisfies *zero-knowledge* we call it a zkSNARK.

- $\mathsf{KeyGen}(R \in \mathcal{R}_\lambda) \rightarrow \mathsf{crs} = (\mathsf{ek}, \mathsf{vk})$: outputs a common reference string consisting of an evaluation and a verification key.
- $\mathsf{Prove}(\mathsf{ek}, x, w) \rightarrow \pi$: returns proof that $R(x, w)$ holds.
- $\mathsf{VerProof}(\mathsf{vk}, x, \pi) \rightarrow b \in \{0 = \textit{reject}, 1 = \textit{accept}\}$

[5] We stress that only our concrete instantiations require a trusted setup—our general composition framework does not.

Completeness. For any pair (x, w) satisfying the relation, the verifier always accepts the corresponding proof.

Succinctness. Π is said *succinct* if the running time of VerProof is $\mathsf{poly}(\lambda)(\lambda + |x| + \log|w|)$ and the proof size is $\mathsf{poly}(\lambda)(\lambda + \log|w|)$.

Knowledge Soundness. Π is $\mathsf{KSND}(\mathcal{RG}, \mathcal{Z})$ if there exist benign relation generator $\mathcal{RG}$ and auxiliary input generator $\mathcal{Z}$ such that for every non-uniform adversary $\mathcal{A}$ there is a non-uniform extractor $\mathcal{E}$ winning the following game with negligible probability:

$$\mathsf{Game}^{\mathsf{KSND}}_{\mathcal{RG},\mathcal{Z},\mathcal{A},\mathcal{E}} \to b$$

$$(R, \mathsf{aux}_R) \leftarrow \mathcal{RG}(1^\lambda) \ ; \ \mathsf{crs} := (\mathsf{ek}, \mathsf{vk}) \leftarrow \mathsf{KeyGen}(R)$$

$$\mathsf{aux}_Z \leftarrow \mathcal{Z}(R, \mathsf{aux}_R, \mathsf{crs}) \ ; \ (x, \pi) \leftarrow \mathcal{A}(R, \mathsf{crs}, \mathsf{aux}_R, \mathsf{aux}_Z)$$

$$w \leftarrow \mathcal{E}(R, \mathsf{crs}, \mathsf{aux}_R, \mathsf{aux}_Z) \ ; \ b = \mathsf{VerProof}(\mathsf{vk}, x, \pi) \wedge \neg R(x, w)$$

Composable Zero-Knowledge. Π is $\mathsf{CZK}(\mathcal{RG})$ if there exists a pair of simulators $\mathcal{S} = (\mathcal{S}_{\mathsf{kg}}, \mathcal{S}_{\mathsf{prv}})$ such that any adversary cannot tell if it receives honestly generated $\mathsf{crs} \leftarrow \mathsf{KeyGen}(R)$ and proofs $\pi \leftarrow \mathsf{Prove}(\mathsf{ek}, x, w)$, or a simulated $(\mathsf{crs}, \mathsf{td}_\mathsf{k}) \leftarrow \mathcal{S}_{\mathsf{kg}}(R)$ and simulated proofs $\pi \leftarrow \mathcal{S}_{\mathsf{prv}}(\mathsf{crs}, \mathsf{td}_\mathsf{k}, x)$.

2.1.1 zkSNARKs with specializable universal CRS.

In [40] Groth et al. introduce a variant of the SNARK notion where Π works for universal relations, the keys output by KeyGen work for the whole family $\mathcal{R}_\lambda$ and can be used to prove and verify statements about any $R \in \mathcal{R}_\lambda$. This is convenient because the cost of setup can be amortized. In a nutshell, this notion formalizes the idea that key generation for R can be seen as the sequential combination of two steps: a first probabilistic algorithm that generates a CRS for the universal relation, and a second *deterministic* algorithm, Derive, that specializes this universal CRS into one for a specific R. We remark that our UC SNARKS follow this model.

3 BUILDING THE LEGOSNARK FRAMEWORK

3.1 Commit and Prove SNARKs

In a nutshell, a *commit-and-prove SNARK* (CP-SNARK) is a SNARK that can prove knowledge of (x, w) such that $R(x, w)$ holds w.r.t. a witness $w = (u, \omega)$ and u opens a commitment c_u.[6] Our formal definitions below add some syntactic sugar to this idea to explicitly handle relations where the input domain $\mathcal{D}_u$ is more fine grained and splits over ℓ subdomains. For reasons that will shortly become clear, we call these subdomains *commitment slots*. This splitting is often natural (e.g., if u is a binary string, one can think of $u := (u_1, \ldots, u_\ell)$ for suitable substrings), and it is crucial to exploit the compositional power of CP-SNARKs, as we show in Section 3.2. We assume the description of the splitting is part of R's description.

Definition 3.1 (CP-SNARKs). Let $\{\mathcal{R}_\lambda\}_{\lambda\in\mathbb{N}}$ be a family of relations R over $\mathcal{D}_x \times \mathcal{D}_u \times \mathcal{D}_\omega$ such that $\mathcal{D}_u$ splits over ℓ arbitrary domains $(\mathcal{D}_1 \times \cdots \times \mathcal{D}_\ell)$ for some arity parameter $\ell \geq 1$. Let $\mathsf{Com} = (\mathsf{Setup}, \mathsf{Commit}, \mathsf{VerCommit})$ be a commitment scheme (as per Definition 2.1) whose input space $\mathcal{D}$ is such that $\mathcal{D}_i \subset \mathcal{D}$ for all $i \in [\ell]$. A commit and prove zkSNARK for Com and $\{\mathcal{R}_\lambda\}_{\lambda\in\mathbb{N}}$ is a zkSNARK for a family of relations $\{\mathcal{R}^{\mathsf{Com}}_\lambda\}_{\lambda\in\mathbb{N}}$ such that:

- every $\mathbf{R} \in \mathcal{R}^{\mathsf{Com}}$ is represented by a pair (ck, R) where $\mathsf{ck} \in \mathsf{Setup}(1^\lambda)$ and $R \in \mathcal{R}_\lambda$;
- $\mathbf{R}$ is over pairs $(\mathbf{x}, \mathbf{w})$ where the statement is $\mathbf{x} := (x, (c_j)_{j\in[\ell]}) \in \mathcal{D}_x \times C^\ell$, the witness is $\mathbf{w} := ((u_j)_{j\in[\ell]}, (o_j)_{j\in[\ell]}, \omega) \in \mathcal{D}_1 \times \cdots \times \mathcal{D}_\ell \times \mathcal{O}^\ell \times \mathcal{D}_\omega$, and the relation $\mathbf{R}$ holds iff $\bigwedge_{j\in[\ell]} \mathsf{VerCommit}(\mathsf{ck}, c_j, u_j, o_j) = 1 \wedge R(x, (u_j)_{j\in[\ell]}, \omega) = 1$

Furthermore, when we say that CP is knowledge-sound for a relation generator $\mathcal{RG}$ and auxiliary input generator $\mathcal{Z}$ (denoted $\mathsf{KSND}(\mathcal{RG}, \mathcal{Z})$, for short) we mean it is a knowledge-sound SNARK for the relation generator $\mathcal{RG}_{\mathsf{Com}}(1^\lambda)$ that runs $\mathsf{ck} \leftarrow \mathsf{Setup}(1^\lambda)$ and $(R, \mathsf{aux}_R) \leftarrow \mathcal{RG}(1^\lambda)$, and returns $((\mathsf{ck}, R), \mathsf{aux}_R)$.

We denote a CP-SNARK as a tuple of algorithms CP = (KeyGen, Prove, VerProof). For ease of exposition, in our constructions we adopt a more explicit syntax for CP's algorithms as defined below.

- $\mathsf{KeyGen}(\mathsf{ck}, R) \to \mathsf{crs} := (\mathsf{ek}, \mathsf{vk})$
- $\mathsf{Prove}(\mathsf{ek}, x, (c_j)_{j\in[\ell]}, (u_j)_{j\in[\ell]}, (o_j)_{j\in[\ell]}, \omega) \to \pi$
- $\mathsf{VerProof}(\mathsf{vk}, x, (c_j)_{j\in[\ell]}, \pi) \to b \in \{0, 1\}$

Remark 1 (Comparing with existing definitions). *The Geppetto paper also defines [26] a notion of commit-and-prove SNARKs. Here we highlight the main differences between their definition and ours. First, our commitment key can be generated without fixing a priori a relation (or a set of them, e.g., a multi-QAP). Second, in their model one needs to commit to data using a commitment key corresponding to a specific portion of the input (in their lingo a "bank"), whereas in our model one can just commit to a vector of data, and only at proving time one assigns that data to a specific input slot. Third, we do not require the commitments does not need to have a trapdoor. Ours is closer to Lipmaa's[49] (although [49] uses trapdoor commitments) and is in fact a specialized SNARK notion when considering relation families including verifying openings of commitments.*

3.2 Composition Properties of CP-SNARKs

In this section, we formally show how the commit-and-prove capability can be used to combine different CP-SNARKs securely.

Conjunction of relations with shared inputs. Let $\{\mathcal{R}^{(0)}_\lambda\}_{\lambda\in\mathbb{N}}$ and $\{\mathcal{R}^{(1)}_\lambda\}_{\lambda\in\mathbb{N}}$ be two families of relations such that, for every $\lambda \in \mathbb{N}$ the input domains $\mathcal{D}^{(0)}_u$ and $\mathcal{D}^{(1)}_u$ of relations $R_0 \in \mathcal{R}^{(0)}_\lambda$ and $R_1 \in \mathcal{R}^{(1)}_\lambda$ respectively can split as follows: $\mathcal{D}^{(0)}_u := \mathcal{D}_0 \times \mathcal{D}_2$ and $\mathcal{D}^{(1)}_u := \mathcal{D}_1 \times \mathcal{D}'_2$ with $\mathcal{D}_2 = \mathcal{D}'_2$.[7] In other words we require these relations to share a commitment slot that we call the *shared slot*. Given the above relation families, we define $\{\mathcal{R}^\wedge_\lambda\}_{\lambda\in\mathbb{N}}$ as the family of relations where for every $\lambda \in \mathbb{N}, \mathcal{R}^\wedge_\lambda = \{R^\wedge_{R_0,R_1} : R_0 \in \mathcal{R}^{(0)}_\lambda, R_1 \in \mathcal{R}^{(1)}_\lambda\}$ and $R^\wedge_{R_0,R_1}(x_0, x_1, u_0, u_1, u_2, (w_0, w_1))$ is defined as the conjunction $R_0(x_0, u_0, u_2, w_0) \wedge R_1(x_1, u_1, u_2, w_1)$.

Let Com be a commitment scheme, for $b \in \{0, 1\}$ let CP_b be a CP-SNARK for Com and $\{\mathcal{R}^{(b)}_\lambda\}_{\lambda\in\mathbb{N}}$. From these, we show how to generically build a CP-SNARK $\mathsf{CP}^\wedge$ for Com and $\{\mathcal{R}^\wedge_\lambda\}_{\lambda\in\mathbb{N}}$. The idea is simple: from the definition of $R^\wedge_{R_0,R_1}$ it is enough to use CP_b to prove and verify the two statements $(x_b, u_b, u_2, w_b)_{b\in\{0,1\}}$ using the same commitment to u_2. Our transformation also guarantees that if both CP_0 and CP_1 are CP-SNARKs with specializable universal CRS, then so is the resulting $\mathsf{CP}^\wedge$. This composition result is formalized in the following theorem. Its proof and a full description of the scheme $\mathsf{CP}^\wedge$ are in Appendix **??**. The main idea behind

[6] Our notion assumes that only a portion of the witness is explicitly committed in c_u.

[7] Note such a splitting is rather general, as $\mathcal{D}_2$ and $\mathcal{D}'_2$, or $\mathcal{D}_0$, or $\mathcal{D}_1$ may be empty.

$$\frac{\mathsf{CP}^{\wedge}.\mathsf{KeyGen}(\mathsf{ck}, R^{\wedge}_{R_0,R_1}) \to (\mathsf{ek}^*, \mathsf{vk}^*)}{\{(\mathsf{ek}_b, \mathsf{vk}_b) \leftarrow \mathsf{CP}_b.\mathsf{KeyGen}(\mathsf{ck}, R_b)\}_{b\in\{0,1\}}}$$

$$\mathsf{ek}^* := (\mathsf{ek}_b)_{b\in\{0,1\}} \ ; \ \mathsf{vk}^* := (\mathsf{vk}_b)_{b\in\{0,1\}}$$

$$\frac{\mathsf{CP}^{\wedge}.\mathsf{Prove}(\mathsf{ek}^*, x_0, x_1, (c_j)_{j\in[:3]}, (u_j)_{j\in[:3]}, (o_j)_{j\in[:3]}, \omega_0, \omega_1) :}{\{\pi_b \leftarrow \mathsf{CP}_b.\mathsf{Prove}(\mathsf{ek}_b, x_b, (c_b, c_2), (u_b, u_2), (o_b, o_2), \omega_b)\}_{b\in\{0,1\}}}$$

$$\textbf{return } \pi^* := (\pi_b)_{b\in\{0,1\}}$$

$$\frac{\mathsf{CP}^{\wedge}.\mathsf{VerProof}(\mathsf{vk}^*, x_0, x_1, (c_j)_{j\in[:3]}, \pi^*) \to \mathsf{b}_0 \wedge \mathsf{b}_1}{\{\mathsf{b}_b \leftarrow \mathsf{CP}_b.\mathsf{VerProof}(\mathsf{vk}_b, x_b, (c_b, c_2), \pi_b)\}_{b\in\{0,1\}}}$$

Figure 1: CP-SNARK construction for AND composition

soundness is that, in order for an adversary to break $\mathsf{CP}^{\wedge}$, it must break either one of the two underlying schemes, $\mathsf{CP}_0, \mathsf{CP}_1$, or the binding of the commitment scheme.

Theorem 3.2. *If* Com *is a computationally binding commitment and, for* $b = 0, 1$, CP_b *is a zkCP-SNARK for* Com *and relation family* $\{\mathcal{R}^{(b)}_{\lambda}\}_{\lambda\in\mathbb{N}}$, *then there is a zkCP-SNARK* $\mathsf{CP}^{\wedge}$ *for* Com *and* $\{\mathcal{R}^{\wedge}_{\lambda}\}_{\lambda\in\mathbb{N}}$.

Functions composition. A CP-SNARK for conjunction of relations can be easily used for proving correctness of *composed functions*, e.g., proving that $\exists(y, w) : z = f(x, y, w)$, where $f(x, y, w) := h(g(x, w), y)$. Indeed, let $R_h(x', y, z) = 1$ iff $\exists(x', y) : h(x', y) = z$, and $R_g(x, x') = 1$ iff $\exists(x', w) : g(x, w) = x'$, then $\exists\,(y, w) : z = f(x, y, w)$ can be expressed as $R_h(x', y, z) \wedge R_g(x, x')$.

Disjunction of relations with shared inputs. We can reduce the case of OR composition to the conjunction construction above. For this we assume relations are defined over elements of a ring. For a relation $R(u)$ denote by $\hat{R}(u, t)$ the relation such that $\hat{R}(u, 0) = 1$ iff $R(u) = 1$ and $\hat{R}(u, t) = 1$ iff $R(u) = 0$ when $t \neq 0$. We can now express the disjunction of $R_0(u_0)$, $R_1(u_1)$ as $R^{\vee}_{R_0,R_1}(u_0, u_1, t_0, t_1) := \hat{R}_0(u_0, t_0) \wedge \hat{R}_1(u_1, t_1) \wedge t_0 t_1 = 0$. For this approach to work we need the proof systems for the two relations R_0, R_1 to support their modified version $\hat{R}_0, \hat{R}_1$, which is the case for proof systems supporting general arithmetic or boolean circuits. Finally, we need a simple efficient proof system for the relation $R_{\mathsf{mul}}(t_0, t_1) = 1$ iff $t_0 t_1 = 0$, where both t_0 and t_1 are committed in two different slots.

Composing more than two relations. We can apply Theorem 3.2 several times to build CP-SNARKs for the composition of different relations (or even the same relation with itself). Succinctness degrades linearly with the number of relations involved into a statement and is preserved if one composes a constant or logarithmic number of relations. This is arguably the case when we deal with heterogeneous computations.

3.3 Commit-Carrying SNARKs

In this section we define a variant of SNARKs that lies in between standard SNARKs and CP-SNARKs. We call these schemes *SNARKs with commit-carrying proofs* (or commit-carrying SNARKs, cc-SNARKs for short). In a nutshell, a cc-SNARK is like a SNARK in which the proof contains a commitment to the portion u of the witness. Essentially the difference is that in cc-SNARKs we assume the extractor outputs the opening of the commitment returned along with the proof. Formalizing this idea requires to make explicit the commitment scheme associated to the SNARK, as well as the commitment key that is part of the common reference string. In the next section we discuss how many of the existing SNARK constructions satisfy this property. Later, in Section 3.5 we show that cc-SNARKs can be lifted to become full fledged, composable, CP-SNARKs. These two results together allow us to compose several existing SNARKs. We define commit-carrying SNARKs as follows:

Definition 3.3 (cc-SNARK). A *commit-carrying zkSNARKs* for $\{\mathcal{R}_{\lambda}\}_{\lambda\in\mathbb{N}}$ is a tuple of algorithms $\mathsf{ccΠ} = (\mathsf{KeyGen}, \mathsf{Prove}, \mathsf{VerProof}, \mathsf{VerCommit})$ that work as follows and satisfy the notions of *completeness*, *succinctness*, *knowledge soundness*, *zero knowledge* and *binding* as defined below.

- $\mathsf{KeyGen}(R) \to (\mathsf{ck}, \mathsf{ek}, \mathsf{vk})$: the key generation takes as input the security parameter λ and a relation $R \in \mathcal{R}_{\lambda}$, and outputs a common reference string that includes a commitment key, an evaluaton key and verification key.
- $\mathsf{Prove}(\mathsf{ek}, x, w) \to (c, \pi; o)$: the proving algorithm takes as input an evaluation key, a statement x and a witness $w := (u, \omega)$ such that the relation $R(x, u, \omega)$ holds, and it outputs a proof π, a commitment c and opening o such that $\mathsf{VerCommit}(\mathsf{ck}, c, u, o) = 1$.
- $\mathsf{VerProof}(\mathsf{vk}, x, c, \pi) \to b$: the verification algorithm takes a verification key, a statement x, a commitment c, and either accepts ($b = 1$) or rejects ($b = 0$) the proof π.
- $\mathsf{VerCommit}(\mathsf{ck}, c, u, o) \to b$: the commitment verification algorithm takes as input a commitment key, a commitment c, a message u and an opening o and accepts ($b = 1$) or rejects ($b = 0$).

cc-SNARKs can be seen as a less versatile version of CP-SNARKs (clearly, a CP-SNARK implies a cc-SNARK). In a cc-SNARK the commitment key depends on the relation taken by KeyGen, and a commitment is freshly created by the Prove algorithm and is tied to a single proof; in a CP-SNARK the commitment key is independent of relations and commitments can also be created independently and shared across different proofs. In the literature, there are examples of schemes that lie in between our notions of CP-SNARK and cc-SNARK; this is the case for commit and prove SNARKs in which the commitment key is relation-dependent, e.g., [26, 62].

Completeness. For any $\lambda \in \mathbb{N}$, $R \in \mathcal{R}_{\lambda}$ and $R(x, w) = 1$, it holds

$$\Pr\begin{bmatrix}(\mathsf{ck}, \mathsf{ek}, \mathsf{vk}) \leftarrow \mathsf{KeyGen}(R); \\ (c, \pi; o) \leftarrow \mathsf{Prove}(\mathsf{ek}, x, w)\end{bmatrix} : \ \mathsf{VerProof}(\mathsf{vk}, x, c, \pi)\Bigg] = 1$$

Succinctness. ccΠ is said *succinct* if the running time of VerProof is $\mathsf{poly}(\lambda)\,(\lambda + |x| + \log|w|)$ and the size of the proof is $\mathsf{poly}(\lambda) \cdot (\lambda + \log|w|)$.

Knowledge Soundness. Let $\mathcal{RG}$ be a relation generator such that $\mathcal{RG}_{\lambda} \subseteq \mathcal{R}_{\lambda}$. ccΠ satisfies knowledge soundness for $\mathcal{RG}$ and auxiliary input distribution $\mathcal{Z}$, or $\mathsf{ccKSND}(\mathcal{RG}, \mathcal{Z})$, if for every (non-uniform) efficient adversary $\mathcal{A}$ there exists a (non-uniform) efficient extractor $\mathcal{E}$ such that $\Pr[\mathsf{Game}^{\mathsf{ccKSND}}_{\mathcal{RG},\mathcal{Z},\mathcal{A},\mathcal{E}} = 1] = \mathsf{negl}$. We say that ccΠ is knowledge sound if there exist benign $\mathcal{RG}$ and $\mathcal{Z}$ such that ccΠ is $\mathsf{ccKSND}(\mathcal{RG}, \mathcal{Z})$.

$$\frac{\mathsf{Game}^{\mathsf{ccKSND}}_{\mathcal{RG},\mathcal{Z},\mathcal{A},\mathcal{E}} \to b \in \{0, 1\}}{(R, \mathsf{aux}_R) \leftarrow \mathcal{RG}(1^{\lambda}) \ ; \ \mathsf{crs} := (\mathsf{ck}, \mathsf{ek}, \mathsf{vk}) \leftarrow \mathsf{KeyGen}(R)}$$

$$\mathsf{aux}_Z \leftarrow \mathcal{Z}(R, \mathsf{aux}_R, \mathsf{crs}) \ ; \ (x, c, \pi) \leftarrow \mathcal{A}(R, \mathsf{crs}, \mathsf{aux}_R, \mathsf{aux}_Z)$$

$$(u, o, \omega) \leftarrow \mathcal{E}(R, \mathsf{crs}, \mathsf{aux}_R, \mathsf{aux}_Z)$$

$$b \leftarrow \mathsf{VerProof}(\mathsf{vk}, x, c, \pi) \wedge \neg(\mathsf{VerCommit}(\mathsf{ck}, c, u, o) \wedge R(x, u, \omega))$$

Composable Zero-Knowledge. A scheme ccΠ has composable zero-knowledge for a relation generator $\mathcal{RG}$ if for every adversary $\mathcal{A}$ there exists a simulator $\mathcal{S} = (\mathcal{S}_{\mathsf{kg}}, \mathcal{S}_{\mathsf{prv}})$ such that both following conditions hold for all adversaries $\mathcal{A}$:

Keys Indistinguishability.

$$\Pr\begin{bmatrix}(R, \mathsf{aux}_R) \leftarrow \mathcal{RG}(1^\lambda); \\ \mathsf{crs} \leftarrow \mathsf{KeyGen}(R) \ : \\ \mathcal{A}(\mathsf{crs}, \mathsf{aux}_R) = 1\end{bmatrix} \approx \Pr\begin{bmatrix}(R, \mathsf{aux}_R) \leftarrow \mathcal{RG}(1^\lambda); \\ (\mathsf{crs}, \mathsf{td}_\mathsf{k}) \leftarrow \mathcal{S}_{\mathsf{kg}}(R) \ : \\ \mathcal{A}(\mathsf{crs}, \mathsf{aux}_R) = 1\end{bmatrix}$$

Proof Indistinguishability. For all (x, w),

$$\Pr\begin{bmatrix}(R, \mathsf{aux}_R) \leftarrow \mathcal{RG}(1^\lambda); & \\ (\mathsf{crs}, \mathsf{td}_\mathsf{k}) \leftarrow \mathcal{S}_{\mathsf{kg}}(R); & : \ \mathcal{A}(\mathsf{crs}, \mathsf{aux}_R, c, \pi) = 1 \wedge \\ (c, \pi; o) \leftarrow \mathsf{Prove}(\mathsf{ek}, x, w) & R(x, w) = 1\end{bmatrix}$$

$$\approx \Pr\begin{bmatrix}(R, \mathsf{aux}_R) \leftarrow \mathcal{RG}(1^\lambda); & \\ (\mathsf{crs}, \mathsf{td}_\mathsf{k}) \leftarrow \mathcal{S}_{\mathsf{kg}}(R); & : \ \mathcal{A}(\mathsf{crs}, \mathsf{aux}_R, c, \pi) = 1 \wedge \\ (c, \pi) \leftarrow \mathcal{S}_{\mathsf{prv}}(\mathsf{crs}, \mathsf{td}_\mathsf{k}, x) & R(x, w) = 1\end{bmatrix}$$

Binding. For every polynomial-time adversary $\mathcal{A}$ the following probability is $\mathsf{negl}(\lambda)$:

$$\Pr\begin{bmatrix}(R, \mathsf{aux}_R) \leftarrow \mathcal{RG}(1^\lambda) & \mathsf{VerCommit}(\mathsf{ck}, c, u', o') \\ \mathsf{crs} := (\mathsf{ck}, \mathsf{ek}, \mathsf{vk}) \leftarrow \mathsf{KeyGen}(R) \ : & \wedge \mathsf{VerCommit}(\mathsf{ck}, c, u, o) \\ (c, u, o, u', o') \leftarrow \mathcal{A}(R, \mathsf{crs}, \mathsf{aux}_R) & \wedge u \neq u'\end{bmatrix}$$

Remark 2. *While our definitions consider the case where the proof contains a commitment to a portion u of the witness $w = (u, \omega)$, notice that this partition of the witness is arbitrary and thus this notion also captures those constructions where the commitment is to the entire witness if one thinks of a void ω.*

3.3.1 cc-SNARKs with Weak Binding.

Let us now define a weaker variant of cc-SNARKs that differs from the one given in Definition 3.3 in that the underlying commitment scheme is not binding in the usual sense. Slightly more in detail, we consider the case where the commitment refers to the whole witness (i.e., ω is an empty string) and it is actually possible to find collisions for a given commitment as long as these collisions are among valid witnesses, or more precisely we require to be computationally infeasible to find two different witnesses that validly open the commitment such that one falsifies the relation and the other one satisfies it. Worth noting that our generic compiler can also turn weak cc-SNARKs into CP-SNARKs.

Definition 3.4 (cc-SNARKs with Weak Binding). We define cc-SNARKs with Weak Binding as in Definition 3.3 with two exceptions: we assume that the scheme is defined only for relations such that $\mathcal{D}_\omega = \emptyset$; we replace the binding property with the one below.

Weak Binding. For every polynomial-time adversary $\mathcal{A}$ the following probability is $\mathsf{negl}(\lambda)$:

$$\Pr\begin{bmatrix}(R, \mathsf{aux}_R) \leftarrow \mathcal{RG}(1^\lambda) & \mathsf{VerCommit}(\mathsf{ck}, c, u, o) \\ \mathsf{crs} := (\mathsf{ck}, \mathsf{ek}, \mathsf{vk}) \leftarrow \mathsf{KG}(R) & \wedge \mathsf{VerCommit}(\mathsf{ck}, c, u', o') \\ (x, c, u, o, u', o', \pi) & \wedge \mathsf{VerProof}(\mathsf{vk}, x, c, \pi) \\ \leftarrow \mathcal{A}(R, \mathsf{crs}, \mathsf{aux}_R) & \wedge u \neq u' \wedge \neg R(x, u) \wedge R(x, u')\end{bmatrix}$$

3.4 Existing CP-SNARKs and cc-SNARKs

In this section, we provide a summary of existing schemes that can be explained, with no or little modification, under our CP-SNARK and cc-SNARK notions. In fact, existing QAP-based schemes [13, 39, 55] are not fully binding but can satisfy our weak binding. In the full version, we prove that [39] is a *weak* cc-SNARK.

Existing CP-SNARKs. The following list is a summary. Details supporting the following claims appear in the full version [21].

- Adaptive Pinocchio [62] is a CP-SNARK for relations $R_Q(x, (u_j)_{j\in[\ell]}, \omega)$ where R_Q is a quadratic arithmetic program (QAP), and the commitment scheme is the extended Pedersen commitment of Groth [38] in which the ith basis of the commitment key is g^{x^i} for a random x.
- The scheme in [49][Section 4] is a CP-SNARK for Hadamard product relations $R^{\mathsf{had}}(\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c})$ over $\mathbb{Z}_q^{3m}$, i.e. R^{had} holds iff $\forall i \in [m] : a_i \cdot b_i = c_i$. In this case the commitment scheme is a variant of the extended Pedersen scheme where the ith basis of the commitment key is $g^{\ell_i(x)}$ for a random x and ℓ_i being the i-th Lagrange basis polynomial.
- zk-vSQL [68] is a CP-SNARK for relations $R((u_j)_{j\in[\ell]})$ where R is an arithmetic circuit, and the commitment is a polynomial commitment that can also be explained as a variant of extended Pedersen (as observed in the full version).

Existing cc-SNARKs. Geppetto [26] is a commit-and-prove SNARK for QAP relations $R_Q(x, u, \omega)$, with a *relation-dependent* commitment key. This scheme immediately yields a cc-SNARK where VerCommit is also a variant of extended Pedersen.

A New Efficient cc-SNARK for QAPs. We show that the SNARK of [39] can be modified to obtain a cc-SNARK for QAP relations $R_Q(u, \omega)$, where the witness portion committed in a fully binding way can be chosen (see full version). Compared to the other cc-SNARKs for QAPs mentioned above, this scheme offers nearly optimal efficiency (essentially due to the fact that we start from [39] whereas [26, 62] build on [55]).

3.5 Bootstrapping our Framework

A key requirement to apply the composition results of the Lego-SNARK framework is to start from CP-SNARKs that share the same commitment scheme. In practice this is not always the case (see for example the discussion in the previous section). In this section we propose a solution to this issue by giving a generic compiler for turning a cc-SNARK ccΠ for a family of relations $\{\mathcal{R}_\lambda\}_{\lambda\in\mathbb{N}}$ into a CP-SNARK CP that supports the same relations and works for a given, global, commitment scheme Com. Incidentally, since a CP-SNARK CP for commitment Com′ is also a cc-SNARK, our compiler can also turn CP into a CP-SNARK for another commitment Com.

As noted in the introduction one could solve this problem via a folklore approach that affects efficiency and requires ccΠ to be expressive enough (which may not be true if ccΠ supports only specialized tasks, as per our vision). Our compiler uses an alternative approach. The intuition is the following. Consider a relation $R(u, w)$ and denote by $c(u)$ a relation-independent commitment to u. A cc-SNARK $\mathsf{cc}\Pi_R$ could prove $R(u, w)$ by producing $(c_R(u), \pi_R)$; this proof however lacks of any link with the commitment $c(u)$. To solve this, we use a specialized CP-SNARK $\mathsf{CP}_{\mathsf{link}}$ for statements

$$\frac{\mathsf{CP.KeyGen}(\mathsf{ck}, R) \to (\mathsf{ek} := (\mathsf{ck}', \mathsf{ek}', \mathsf{ek}^\circ), \mathsf{vk} := (\mathsf{vk}', \mathsf{vk}^\circ))}{\begin{array}{l}(\mathsf{ck}', \mathsf{ek}', \mathsf{vk}') \leftarrow \mathsf{cc\Pi.KeyGen}(R); \text{Build } R^\circ \text{ from } (\mathsf{ck}', \mathcal{D}^\circ_x, \mathcal{D}^\circ_u, \mathcal{D}^\circ_\omega) \\ (\mathsf{ek}^\circ, \mathsf{vk}^\circ) \leftarrow \mathsf{CP}_{\mathsf{link}}.\mathsf{KeyGen}(\mathsf{ck}, R^\circ)\end{array}}$$

$$\frac{\mathsf{CP.Prove}(\mathsf{ek}, x, (c_j, u_j, o_j)_{j\in[\ell]}, \omega) \to \pi := (c', \pi^\circ, \pi')}{\begin{array}{l}(c', \pi', o') \leftarrow \mathsf{cc\Pi.Prove}(\mathsf{ek}', x, (u_j)_{j\in[\ell]}; \omega);\ (x^\circ, \omega^\circ) := (c', o') \\ \pi^\circ \leftarrow \mathsf{CP}_{\mathsf{link}}.\mathsf{Prove}(\mathsf{ek}^\circ, x^\circ, (c_j)_{j\in[\ell]}, (u_j)_{j\in[\ell]}, (o_j)_{j\in[\ell]}, \omega^\circ)\end{array}}$$

$$\frac{\mathsf{CP.VerProof}(\mathsf{vk}, x, (c_j)_{j\in[\ell]}, \pi) \to \{0,1\}}{\mathsf{CP}_{\mathsf{link}}.\mathsf{VerProof}(\mathsf{vk}^\circ, c', (c_j)_{j\in[\ell]}, \pi^\circ) \wedge \mathsf{cc\Pi.VerProof}(\mathsf{vk}', x, c', \pi')}$$

Figure 2: Generic Construction of CP from $\mathsf{CP}_{\mathsf{link}}$ and ccΠ.

like "$c(u)$ and $c_R(u)$ open to the same value", which can be seen as a commit-and-prove relation about the opening of c_R with respect to a global commitment c. This minimal tool turns any cc-SNARK into a full fledged CP-SNARK. Despite the funny fact that we require a CP-SNARK to create CP-SNARKs, $\mathsf{CP}_{\mathsf{link}}$ can be a simple scheme (as confirmed by our construction in Section 4.1) so it allows us to create efficient instantiations of CP-SNARKs.

Our cc-SNARK-lifting compiler. Let ccΠ be a cc-SNARK for a family of relations $\{\mathcal{R}_\lambda\}_{\lambda\in\mathbb{N}}$ where, for every λ, $R \in \mathcal{R}_\lambda$ is over tuples in $\mathcal{D}_x \times \mathcal{D}_u \times \mathcal{D}_\omega$, and $\mathcal{D}_u$ splits over ℓ subdomains $(\mathcal{D}_1 \times \cdots \times \mathcal{D}_\ell)$ for some arity parameter ℓ. Consider the commitment verification algorithm ccΠ.VerCommit. For any $\lambda \in \mathbb{N}$ and any $\mathsf{ck}' \in \{\mathsf{cc\Pi.KeyGen}(R)\}_{R\in\mathcal{R}_\lambda}$, we define the relation R° that has input space $\mathcal{D}^\circ_x = C'$, and witness space $\mathcal{D}^\circ_\omega = \mathcal{D}^\circ_u \times \mathcal{D}^\circ_\omega$ such that $\mathcal{D}^\circ_u = \mathcal{D}_1 \times \cdots \times \mathcal{D}_\ell$ and $\mathcal{D}^\circ_\omega := O'$, where C' and O' are the commitment and opening space of the commitment of ccΠ. For compactness we represent R° with $(\mathsf{ck}', \mathcal{D}^\circ_x, \mathcal{D}^\circ_u, \mathcal{D}^\circ_\omega)$. Then, R° is defined as follows:

$$R^\circ\left(x^\circ, (u^\circ_j)_{j\in[\ell]}, \omega^\circ\right) := \mathsf{cc\Pi.VerCommit}(\mathsf{ck}', x^\circ, (u^\circ_j)_{j\in[\ell]}, \omega^\circ)$$

We remark that, above, $x^\circ \in C'$ is a commitment for ccΠ.VerCommit and $\omega^\circ \in O'$ is its opening.

Let $\mathsf{CP}_{\mathsf{link}}$ be a CP-SNARK for Com and a family of relations $\{\mathcal{R}^\circ_\lambda\}_{\lambda\in\mathbb{N}}$ such that for every $\lambda \in \mathbb{N}$ the relation R° defined above is in $\mathcal{R}^\circ_\lambda$. In Figure 2 we describe a CP-SNARK CP for $\{\mathcal{R}_\lambda\}_{\lambda\in\mathbb{N}}$ that works by using ccΠ and $\mathsf{CP}_{\mathsf{link}}$.

We state the following result about the security of our compiler.

Theorem 3.5. *If ccΠ is a zk-cc-SNARK (or a weak cc-SNARK) for $\{\mathcal{R}_\lambda\}_{\lambda\in\mathbb{N}}$ and $\mathsf{CP}_{\mathsf{link}}$ is a zk-CP-SNARK for $\{\mathcal{R}^\circ_\lambda\}_{\lambda\in\mathbb{N}}$, then the scheme CP in Figure 2 is a zk-CP-SNARK for $\{\mathcal{R}_\lambda\}_{\lambda\in\mathbb{N}}$.*

4 CP-SNARKS FOR PEDERSEN-LIKE COMMITMENTS

In this section we propose two CP-SNARKs that work for any commitment scheme whose verification algorithm is the same as the extended Pedersen commitment (essentially a multi-exponentiation). For vectors committed in this way, we show two schemes. Our first scheme (given in Section 4.1) allows to prove that another commitment, with the same verification algorithm but different key, opens to the same vector. This is essentially an efficient realization of the $\mathsf{CP}_{\mathsf{link}}$ CP-SNARK needed in our compiler of Section 3.5, and that works for cc-SNARKs whose underlying commitment verification has the same structure as Pedersen. Our second scheme (given in Section 4.2) instead allows one to prove correctness of a linear function of the committed vector (i.e., that $\boldsymbol{x} = \boldsymbol{F} \cdot \boldsymbol{u}$).

In what follows we start by recalling facts and notation about bilinear groups and the Pedersen commitment.

Preliminaries on Bilinear Groups. A *bilinear group generator* $\mathcal{BG}(1^\lambda)$ outputs $(q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$, where $\mathbb{G}_1$, $\mathbb{G}_2$, $\mathbb{G}_T$ are additive groups of prime order q, and $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is an efficiently computable, non-degenerate, bilinear map. In this paper, we consider Type-3 groups where it is assumed there is no efficiently computable isomorphism between $\mathbb{G}_1$ and $\mathbb{G}_2$. We use bracket notation of [27], i.e., for $s \in \{1, 2, T\}$ and $a \in \mathbb{Z}_q$, we write $[a]_s$ to denote $a \cdot g_s \in \mathbb{G}_s$, where g_s is a fixed generator of $\mathbb{G}_s$. From an element $[a]_s \in \mathbb{G}_s$ and a scalar b it is possible to efficiently compute $[ab] \in \mathbb{G}_s$. Also, given elements $[a]_1 \in \mathbb{G}_1$ and $[b]_2 \in \mathbb{G}_2$, one can efficiently compute $[a \cdot b]_T$ by using the pairing $e([a]_1, [b]_2)$, that we compactly denote with $[a]_1 \cdot [b]_2$. Vectors and matrices are denoted in boldface. We use the bracket notation also for matrix operations, i.e., $[\boldsymbol{A}]_1 \cdot [\boldsymbol{B}]_2 = [\boldsymbol{A} \cdot \boldsymbol{B}]_T$. For a vector $\boldsymbol{a}$ and for $i < j$ we denote by $\boldsymbol{a}_{[i,j]}$ its portion $(a_i, \ldots a_j)$.

Pedersen Vector Commitment. Let us recall the extended Pedersen commitment scheme for vectors of size n. Here we consider an instantiation on a group $\mathbb{G}_1$.

$\mathsf{Ped.Setup}(1^\lambda) \to \mathsf{ck} := [\boldsymbol{h}]_1 \leftarrow \mathbb{G}_1^{n+1}$ from distribution $\mathcal{D}$

$\mathsf{Ped.Commit}([\boldsymbol{h}]_1, \boldsymbol{w}) \to (c, o) := ((o, \boldsymbol{w}^\top) \cdot [\boldsymbol{h}]_1, o)$ where $o \leftarrow\$ \mathbb{Z}_q$

$\mathsf{Ped.VerCommit}([\boldsymbol{h}]_1, c, \boldsymbol{w}, o) \to c \stackrel{?}{=} (o, \boldsymbol{w}^\top) \cdot [\boldsymbol{h}]_1$

Above $\mathcal{D}$ is a probability distribution over the group elements that allows to argue that the scheme is perfectly hiding and computationally binding. For example, $\mathcal{D}$ may be the uniform distribution, in which case we obtain the classical scheme that is binding under the discrete logarithm assumption, or $\mathcal{D}$ may output powers of random values, e.g., $h_i = s^i$ for an $s \leftarrow\$ \mathbb{Z}_q$, that has also been proved computationally binding under a suitable assumption.

In our constructions we only require the commitment scheme to have the same verification algorithm as Ped.VerCommit.

Tool: SNARK for Linear Subspaces. In our CP-SNARK constructions we make use of a SNARK for the linear subspace relation $R_{\mathbf{M}}([\boldsymbol{x}]_1, \boldsymbol{w}) = [\boldsymbol{x}]_1 \stackrel{?}{=} [\mathbf{M}]_1 \cdot \boldsymbol{w} \in \mathbb{G}_1^l$ where $[\mathbf{M}] \in \mathbb{G}_1^{l\times t}, \boldsymbol{w} \in \mathbb{Z}_q^t$. Namely, given a fixed public matrix $[\mathbf{M}]_1$ and a public vector $[\boldsymbol{x}]_1$, one can prove knowledge of a vector $\boldsymbol{w}$ such that $[\boldsymbol{x}]_1 = [\mathbf{M}]_1 \cdot \boldsymbol{w}$. We denote a SNARK for this family of relations with ss″. A candidate scheme for ss″ is the Kiltz-Wee QA-NIZK scheme Π'_{as} [46] that works in bilinear groups. As described in [46], the security of this scheme requires that $l > t$, which is not satisfied in our setting where matrices have always more columns than rows. This means that, when $\mathbf{M}$ has full rank, $R_{\mathbf{M}}$ is satisfied for any $[\boldsymbol{x}]_1$. In fact, what we need is an argument of knowledge for this language. For this, by extending a recent result [28], we show the knowledge soundness of Π'_{as} [46], without the $l > t$ restriction, under the discrete logarithm assumption, in the algebraic group model [30]. For knowledge soundness, the matrix $[\mathbf{M}]_1$ must be generated using a witness sampleable distribution $\mathcal{D}_{\mathsf{mtx}}$, i.e., there must exist a polynomial time algorithm that samples $\mathbf{M}$ in $\mathbb{Z}_q$ such that $[\mathbf{M}]_1$ has the same distribution as the one sampled with $\mathcal{D}_{\mathsf{mtx}}$. We note that this is satisfied by our use cases where $\mathbf{M}$ includes bases of Pedersen-like commitment schemes.

4.1 CP-SNARK for Pedersen Verification

Let Com be a commitment scheme such that Com.VerCommit = Ped.VerCommit. We build a CP-SNARK $\mathsf{CP}_{\mathsf{link}}$ for Com and for the following class of relations R°. Fixed a security parameter λ (and the group setting for λ as well), R° is over $(\mathcal{D}_x \times \mathcal{D}_1 \times \cdots \times \mathcal{D}_\ell \times \mathcal{D}_\omega)$, where $\mathcal{D}_x = \mathbb{G}_1$, $\mathcal{D}_\omega = \mathbb{Z}_q$ and $\mathcal{D}_j = \mathbb{Z}_q^{n_j}$ for some n_j such that $\sum_j n_j = m$. R° is parametrized by a commitment key $[\boldsymbol{f}]_1 \in \mathbb{G}_1^{m+1}$, and is defined as: $R^\circ\big(c', (\boldsymbol{u}_j)_{j\in[\ell]}, o'\big) = 1 \iff c' \stackrel{?}{=} (o', \boldsymbol{u}_1^\top, \ldots, \boldsymbol{u}_\ell^\top) \cdot [\boldsymbol{f}]_1$. Before describing the construction in full detail, let us present the main ideas.

Let $\mathsf{ck} = [\boldsymbol{h}]_1 \in \mathbb{G}_1^{n+1}$ be the key of the global commitment Com. In our $\mathsf{CP}_{\mathsf{link}}$ the public inputs of the prover are ℓ commitments $(c_j)_{j\in[\ell]}$ and another commitment c'; the witness is a set of openings $((\boldsymbol{u}_j)_{j\in[\ell]}, (o_j)_{j\in[\ell]})$ for commitments $(c_j)_{j\in[\ell]}$, and an opening o' for c'. In particular, the prover must prove that

$$R^\circ_{\mathsf{Ped}}(c', (c_j)_{j\in[\ell]}, (\boldsymbol{u}_j)_{j\in[\ell]}, (o_j)_{j\in[\ell]}, o') = 1 \iff \bigwedge\nolimits_{j\in[\ell]} c_j = (o_j, \boldsymbol{u}_j^\top) \cdot [\boldsymbol{h}_{[0,n_j]}]_1 \ \wedge\ c' = (o', \boldsymbol{u}_1^\top, \ldots, \boldsymbol{u}_\ell^\top) \cdot [\boldsymbol{f}]_1$$

The description of our scheme $\mathsf{CP}_{\mathsf{link}}$ follows:

$\mathsf{CP}_{\mathsf{link}}.\mathsf{KeyGen}(\mathsf{ck}, R^\circ)$**:** parse $\mathsf{ck} = [\boldsymbol{h}]_1 \in \mathbb{G}_1^{n+1}$, and let $R^\circ : \mathbb{G}_1 \times \mathcal{D}_1 \times \cdots \times \mathcal{D}_\ell \times \mathbb{Z}_q$ be the relation defined above with $\mathsf{ck}' = [\boldsymbol{f}]_1 \in \mathbb{G}_1^{m+1}$. Use $[\boldsymbol{h}]_1$, $[\boldsymbol{f}]_1$ and R° to build a matrix $\mathbf{M}$ as in equation (1). Compute $(\mathsf{ek}, \mathsf{vk}) \leftarrow \mathsf{ss}''.\mathsf{KeyGen}([\mathbf{M}]_1)$ and return $(\mathsf{ek}, \mathsf{vk})$.

$\mathsf{CP}_{\mathsf{link}}.\mathsf{Prove}(\mathsf{ek}, c', (c_j)_{j\in[\ell]}, (\boldsymbol{u}_j)_{j\in[\ell]}, (o_j)_{j\in[\ell]}, o')$: define $[\boldsymbol{x}]_1$ and $\boldsymbol{w}$ as in as in equation (1), compute $\pi \leftarrow \mathsf{ss}''.\mathsf{Prove}(\mathsf{ek}, [\boldsymbol{x}]_1, \boldsymbol{w})$ and return π.

$\mathsf{CP}_{\mathsf{link}}.\mathsf{VerProof}(\mathsf{vk}, c', (c_j)_{j\in[\ell]}, \pi)$**:** set $[\boldsymbol{x}]_1$ as in (1) and return $\mathsf{ss}''.\mathsf{VerProof}(\mathsf{vk}, [\boldsymbol{x}]_1, \pi)$.

The key idea of the construction is that this relation can be expressed as a linear subspace relation $R_{\mathbf{M}}([\boldsymbol{x}]_1, \boldsymbol{w})$ where $\mathbf{M}, \boldsymbol{x}, \boldsymbol{w}$ can be defined as follows from the inputs of R°_{Ped}, with $l = \ell + 1$ and $t = m + \ell + 1$:

$$\overbrace{\begin{bmatrix} c_1 \\ \vdots \\ c_\ell \\ c' \end{bmatrix}_1}^{[\boldsymbol{x}]_1} = \overbrace{\begin{bmatrix} h_0 & 0 & \ldots & 0 & 0 & \boldsymbol{h}_{[1,n_1]} & 0 & \ldots & 0 \\ 0 & h_0 & \ldots & 0 & 0 & 0 & \boldsymbol{h}_{[1,n_2]} & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & h_0 & 0 & 0 & 0 & \ldots & \boldsymbol{h}_{[1,n_\ell]} \\ 0 & 0 & \ldots & 0 & f_0 & \boldsymbol{f}_{[1,n_1]} & \boldsymbol{f}_{[n_1+1,n_2]} & \cdots & \boldsymbol{f}_{[n_{\ell-1}+1,n_\ell]} \end{bmatrix}_1}^{[\mathbf{M}]_1} \overbrace{\begin{pmatrix} o_1 \\ \vdots \\ o_\ell \\ o' \\ \boldsymbol{u}_1 \\ \vdots \\ \boldsymbol{u}_\ell \end{pmatrix}}^{\boldsymbol{w}} \tag{1}$$

In the following theorem we show that $\mathsf{CP}_{\mathsf{link}}$ is knowledge-sound and zero-knowledge assuming so is ss''. We prove the security of $\mathsf{CP}_{\mathsf{link}}$ based on that of ss'' in the full version, where we also show how to extend $\mathsf{CP}_{\mathsf{link}}$ to handle a more general class of relations that essentially checks that a set of vectors $(\boldsymbol{u}_j)_{j\in[\ell]}$ is a *prefix*, of known length, of a vector $\boldsymbol{u}'$ committed in c'.

Theorem 4.1. *If* ss'' *is* $\mathsf{KSND}(\mathsf{ss}''.\mathcal{RG}, \mathcal{Z})$ *where* $\mathcal{Z}$ *is an auxiliary input distribution, then the CP-SNARK construction* $\mathsf{CP}_{\mathsf{link}}$ *given above is* $\mathsf{KSND}(\mathsf{CP}_{\mathsf{link}}.\mathcal{RG}, \mathcal{Z})$. *Furthermore, if* ss'' *is composable ZK for* $\mathsf{ss}''.\mathcal{RG}$, *then* $\mathsf{CP}_{\mathsf{link}}$ *is composable ZK for* $\mathsf{CP}_{\mathsf{link}}.\mathcal{RG}$.

4.2 CP-SNARK for Linear Properties

In this section we show a CP-SNARK for the relation R^{lin} that checks linear properties of (committed) vectors: for a fixed public matrix $\boldsymbol{F} \in \mathbb{Z}_q^{N\times m}$, relation $R^{\mathsf{lin}}_{\boldsymbol{F}}$ over public input $\boldsymbol{x} \in \mathbb{Z}_q^N$ and witness $\boldsymbol{u} \in \mathbb{Z}_q^m$, with $\boldsymbol{u} := (\boldsymbol{u}_j)_{j\in[\ell]}$ and $\boldsymbol{u}_j \in \mathbb{Z}_q^{n_j}$, holds iff $\boldsymbol{x} \stackrel{?}{=} \boldsymbol{F} \cdot \boldsymbol{u}$.

The scheme, called $\mathsf{CP}^{\mathsf{Ped}}_{\mathsf{lin}}$, is quite similar to $\mathsf{CP}_{\mathsf{link}}$ and essentially consists into invoking ss'' to prove that the above subspace relation holds. The full description of our scheme $\mathsf{CP}^{\mathsf{Ped}}_{\mathsf{lin}}$ follows:

$\mathsf{CP}^{\mathsf{Ped}}_{\mathsf{lin}}.\mathsf{KeyGen}(\mathsf{ck}, R^{\mathsf{lin}}_{\boldsymbol{F}})$**:** parse $\mathsf{ck} = [\boldsymbol{h}]_1 \in \mathbb{G}_1^{m+1}$. Use $[\boldsymbol{h}]_1$ and $R^{\mathsf{lin}}_{\boldsymbol{F}}$ to build a matrix $[\mathbf{M}]$ as in equation (2). Compute $(\mathsf{ek}, \mathsf{vk}) \leftarrow \mathsf{ss}''.\mathsf{KeyGen}([\mathbf{M}]_1)$ and return $(\mathsf{ek}, \mathsf{vk})$.

$\mathsf{CP}^{\mathsf{Ped}}_{\mathsf{lin}}.\mathsf{Prove}(\mathsf{ek}, \boldsymbol{x}, (c_j)_{j\in[\ell]}, (\boldsymbol{u}_j)_{j\in[\ell]}, (o_j)_{j\in[\ell]})$**:** define $[\boldsymbol{x}']_1$ and $\boldsymbol{w}'$ as in equation (2), and return $\pi \leftarrow \mathsf{ss}''.\mathsf{Prove}(\mathsf{ek}, [\boldsymbol{x}']_1, \boldsymbol{w}')$.

$\mathsf{CP}^{\mathsf{Ped}}_{\mathsf{lin}}.\mathsf{VerProof}(\mathsf{vk}, \boldsymbol{x}, (c_j)_{j\in[\ell]}, \pi)$**:** set $[\boldsymbol{x}']_1$ as in (2) and return $\mathsf{ss}''.\mathsf{VerProof}(\mathsf{vk}, [\boldsymbol{x}']_1, \pi)$.

Our scheme $\mathsf{CP}^{\mathsf{Ped}}_{\mathsf{lin}}$ considers each $\boldsymbol{u}_j$ to be committed using a commitment scheme Com such that Com.VerCommit = Ped.VerCommit, and whose key is $\mathsf{ck} = [\boldsymbol{h}]_1 \in \mathbb{G}_1^{m^*+1}$, with $m^* \geq m$.[8] The idea is to express such a commit-and-prove relation with the linear subspace relation $R_{\mathbf{M}}([\boldsymbol{x}']_1, \boldsymbol{w}')$ that holds iff $[\boldsymbol{x}']_1 = [\mathbf{M}]_1 \cdot \boldsymbol{w}'$, where $[\boldsymbol{x}']_1 \in \mathbb{G}_1^l$, $[\mathbf{M}]_1 \in \mathbb{G}_1^{l\times t}$ and $\boldsymbol{w}' \in \mathbb{Z}_q^t$ can be built from the inputs of $R^{\mathsf{lin}}_{\boldsymbol{F}}$ as follows (for $l = \ell + N$ and $t = m + \ell$):

$$\overbrace{\begin{bmatrix} c_1 \\ \vdots \\ c_\ell \\ \boldsymbol{x} \end{bmatrix}_1}^{[\boldsymbol{x}']_1} = \overbrace{\left[\begin{array}{cccc|cccc} h_0 & 0 & \ldots & 0 & \boldsymbol{h}_{[1,n_1]} & 0 & \ldots & 0 \\ 0 & h_0 & \ldots & 0 & 0 & \boldsymbol{h}_{[1,n_2]} & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & h_0 & 0 & 0 & \ldots & \boldsymbol{h}_{[1,n_\ell]} \\ \hline \multicolumn{4}{c|}{\mathbf{0}} & \multicolumn{4}{c}{\boldsymbol{F}} \end{array}\right]_1}^{[\mathbf{M}]_1} \overbrace{\begin{pmatrix} o_1 \\ \vdots \\ o_\ell \\ \boldsymbol{u}_1 \\ \vdots \\ \boldsymbol{u}_\ell \end{pmatrix}}^{\boldsymbol{w}'} \tag{2}$$

Efficiency. When using ss'' from [46], the prover cost is one multi-exponentiation of length $m + \ell$ while the verifier needs $\ell + |\boldsymbol{x}| + 1$ pairings. If $\boldsymbol{x}$ is some fixed value, as in our applications, $|\boldsymbol{x}|$ of these pairings either disappear (if $\boldsymbol{x} = \mathbf{0}$) or can be precomputed. Furthermore, it is possible to see that the cost of KeyGen is $O(\ell \cdot t + n_F)$ where n_F is the number of nonzero entries of $\boldsymbol{F}$. Essentially this cost depends on the sparsity of the matrix; this is crucial in our applications where for example $\boldsymbol{F}$ includes the $\boldsymbol{W}$ matrices representing the linear constraints of a circuit [16].

We state the following theorem. We omit the proof, which is essentially the same as that of Theorem 4.1.

Theorem 4.2. *Let* $\boldsymbol{F} \in \mathbb{Z}_q^{N\times m}$ *be a matrix from a distribution* $\mathcal{D}_{\mathsf{mtx}}$, *and* $\mathcal{Z}$ *be an auxiliary input distribution. If* ss'' *is* $\mathsf{KSND}(\mathsf{ss}''.\mathcal{RG}, \mathcal{Z})$ *where* $\mathsf{ss}''.\mathcal{RG}$ *is a relation generator that samples* ck *and* $\boldsymbol{F} \leftarrow \mathcal{D}_{\mathsf{mtx}}$, *then the CP-SNARK construction* $\mathsf{CP}^{\mathsf{Ped}}_{\mathsf{lin}}$ *given above is* $\mathsf{KSND}(\mathcal{D}_{\mathsf{mtx}}, \mathcal{Z})$. *Furthermore, if* ss'' *is composable ZK for* $\mathsf{ss}''.\mathcal{RG}$, *then* $\mathsf{CP}^{\mathsf{Ped}}_{\mathsf{lin}}$ *is composable ZK for* $\mathcal{D}_{\mathsf{mtx}}$.

[8] While in our description we use the same commitment key for every $\boldsymbol{u}_j$, our scheme easily extends to the case where different commitment keys are used.

5 EFFICIENT CP-SNARKS FOR POLYNOMIAL COMMITMENTS

This section shows a collection of zero-knowledge CP-SNARKs for a variety of relations over vectors committed using a specific polynomial commitment scheme from [68], that we call PolyCom.

	$\mathcal{P}$	$\mathcal{V}$	$\lvert\mathsf{crs}\rvert$	$\lvert\pi\rvert$	
			$\mathbb{G}$	$\mathbb{G}$	$\mathbb{F}$
$\mathsf{CP}_{\mathsf{sc}}$	$O(m)$	$O(\mu)$	$O(\binom{\mu d+\mu}{\mu d})$	$O(d\mu)$	$O(\mu)$
$\mathsf{CP}_{\mathsf{ipd}}$	$O(n)$	$O(\log^2 n)$	$O(n)$	$O(\log^2 n)$	$O(\log^2 n)$
$\mathsf{CP}_{\mathsf{had}}$	$O(n)$	$O(\log n)$	$O(n)$	$O(\log n)$	$O(\log n)$
$\mathsf{CP}_{\mathsf{lin}}$	$O(n^2)$	$O(\log n^2)$	$O(n)$	$O(\log n^2)$	$O(\log n)$
$\mathsf{CP}_{\mathsf{sfprm}}$	$O(n)$	$O(\log^2 n)$	$O(n)$	$O(\log^2 n)$	$O(\log^2 n)$
$\mathsf{CP}_{\mathsf{mm}}$	$O(n^2)$	$O(\log n)$	$O(n^2)$	$O(\log n)$	$O(\log n)$

Table 2: Theoretical results of our CP-SNARKs. In each scheme, n represents: length of vectors in $\mathsf{CP}_{\mathsf{had}}$, $\mathsf{CP}_{\mathsf{ipd}}$ and $\mathsf{CP}_{\mathsf{sfprm}}$; witness length in $\mathsf{CP}_{\mathsf{lin}}$; one dimension of square matrices in $\mathsf{CP}_{\mathsf{mm}}$. In $\mathsf{CP}_{\mathsf{sc}}$, μ represents the number of variables of the polynomials, while d and m are maximum degree and number of monomials respectively.

5.1 Preliminaries and Building Blocks

We review the main building blocks of our constructions.

Polynomial Commitments. We abstract away the VPD primitive of [68] distinguishing between the commitment scheme for multivariate polynomials (that we call PolyCom) and the proof system for committed evaluation at a public point (that we call $\mathsf{CP}_{\mathsf{poly}}$). PolyCom is a *linearly homomorphic extractable trapdoor polynomial commitment scheme* consisting of a tuple of algorithms:

$\mathsf{Setup}(1^\lambda) \to \mathsf{ck}$: outputs a commitment key for a class $\mathcal{F} \subset \mathbb{F}[\boldsymbol{X}]$.

$\mathsf{ComPoly}(\mathsf{ck}, f) \to (c_f, o_f)$: commits a polynomial $f \in \mathcal{F}$.

$\mathsf{ComVal}(\mathsf{ck}, y) \to (c_y, o_y)$: commits a value $y \in \mathbb{F}$.

$\mathsf{CheckCom}(\mathsf{ck}, c) \to b$: takes a commitment and accepts it or not.

$\mathsf{VerCommit}(\mathsf{ck}, c_f, f, o_f) \to b$: accepts or rejects openings.

$\mathsf{HomEval}(\mathsf{ck}, g : \mathbb{F}^\ell \to \mathbb{F}, (c_j)_{j\in[\ell]}, (o_j)_{j\in[\ell]}) \to (c', o')$: computes commitment and opening corresponding to the homomorphic evaluation of a linear function on commitments-openings (c_j, o_j).

Also, denote by $\mathsf{ComPoly}^*$ a version of ComPoly that uses fixed randomness (e.g., 0), and similarly ComVal.

CP-SNARKs for PolyCom**.** We use the following ZK-CP-SNARKs for PolyCom (whose ek is just ck, and vk := cvk, a subset of ck enough to run CheckCom, ComVal and HomEval):

- $\mathsf{CP}_{\mathsf{eq}}$ for the relation $R^{\mathsf{eq}}(u_1, u_2) := u_1 \stackrel{?}{=} u_2 : \forall u_j \in \mathbb{F}$
- $\mathsf{CP}_{\mathsf{prd}}$ for the relation $R^{\mathsf{prd}}(u_1, u_2, u_3) := u_3 \stackrel{?}{=} u_1 \cdot u_2 : \forall u_j \in \mathbb{F}$
- $\mathsf{CP}_{\mathsf{poly}}$ for the relation $R^{\mathsf{poly}}(\boldsymbol{x} \in \mathbb{F}^\mu, f \in \mathcal{F}, y \in \mathbb{F}) := y \stackrel{?}{=} f(\boldsymbol{x})$

The first two can be obtained using classical Sigma protocols; $\mathsf{CP}_{\mathsf{poly}}$ is extracted from [68] and shown in the full version. We recall that $\mathsf{CP}_{\mathsf{poly}}$ proving time is $O(m)$ (with m the number of nonzero monomials of f), whereas verification time and proof size is $O(\mu)$.

Multilinear Extensions. Given a function $f : \{0,1\}^\mu \to \mathbb{F}$, its unique multilinear extension (MLE) is the (unique) multilinear polynomial $\tilde{f} : \mathbb{F}^\mu \to \mathbb{F}$ such that $f(\boldsymbol{b}) = \tilde{f}(\boldsymbol{b})$ for all $\boldsymbol{b} \in \{0,1\}^\mu$. Such multilinear extension is defined as the following polynomial

$$\tilde{f}(X_1, \ldots, X_\mu) = \sum_{\boldsymbol{b}\in\{0,1\}^\mu} \chi_{\boldsymbol{b}}(X_1, \ldots, X_\mu) \cdot f(\boldsymbol{b})$$

where $\chi_{\boldsymbol{b}}(X_1, \ldots, X_\mu) = \prod_{j=1}^{\mu} \chi_{b_j}(X_j)$, $\chi_1(X) = X$ and $\chi_0(X) = 1 - X$. For a vector $\boldsymbol{u} \in \mathbb{F}^m$ (for some $m = 2^\mu$), its unique MLE is the MLE $\tilde{u}$ of the function $u : \{0,1\}^\mu \to \mathbb{F}$ such that, for every $0 \le i \le m-1$ with $i = \sum_{j=0}^{\mu-1} i_j 2^j$, $u(i_0, \ldots, i_{\mu-1}) = u_{i+1}$. Note that by using MLEs one can commit to a vector $\boldsymbol{u}$ using PolyCom by committing to its MLE $\tilde{u}$.

Let $eq : \{0,1\}^\mu \times \{0,1\}^\mu \to \{0,1\}$ be the equality predicate ($eq(a,b) = 1$ iff $a = b$) and let $\widetilde{eq}$ be its MLE (which has a closed-form representation that allows evaluation in time $O(\mu)$ [60]). We recall the following lemma from [58] (as restated in [60]):

Lemma 5.1 ([58, Lemma 3.2.1]). *For any polynomial $h : \mathbb{F}^\mu \to \mathbb{F}$ extending $p : \{0,1\}^\mu \to \mathbb{F}$ (i.e., such that $\forall \boldsymbol{b} \in \{0,1\}^\mu : h(\boldsymbol{b}) = p(\boldsymbol{b})$), it holds that $\tilde{p}(\boldsymbol{X}) = \sum_{\boldsymbol{b}\in\{0,1\}^\mu} \widetilde{eq}(\boldsymbol{X}, \boldsymbol{b}) \cdot h(\boldsymbol{b})$*

5.2 A CP-SNARK for Sum-Check

The sum-check protocol [50] is an interactive proof that allows a prover to convince a verifier of the validity of a statement of the form $t = \sum_{\boldsymbol{b}\in\{0,1\}^\mu} g(\boldsymbol{b})$ where $g : \mathbb{F}^\mu \to \mathbb{F}$. The protocol consists of μ rounds, it is public coin, and the running time of the verifier in it is $O(\sum_{i=1}^{\mu} \deg_i(g))$ plus the cost of evaluating g once (on a random point).

Here we propose a zero-knowledge variant of the sum-check protocol where both the polynomial g and the target value t are committed and one proves knowledge of these values such that $t = \sum_{\boldsymbol{b}\in\{0,1\}^\mu} g(\boldsymbol{b})$. Precisely, we work with polynomials g defined as the product of $p+1$ polynomials of the form $g(\boldsymbol{S}) = \prod_{i=0}^{p} g_i(\boldsymbol{S})$, such that all the g_i's, except g_0, are committed. Namely, we show a CP-SNARK $\mathsf{CP}_{\mathsf{sc}}$ for commitment scheme PolyCom and the relation $R^{\mathsf{sc}}(\boldsymbol{x}, \boldsymbol{u})$, with $\boldsymbol{x} \in \mathcal{F}$ and $\boldsymbol{u} \in \mathbb{F} \times \mathcal{F}^p$, that is formally defined as:

$$R^{\mathsf{sc}}(g_0, (t, (g_j)_{j\in[p]})) = 1 \iff g(\boldsymbol{S}) = \prod_{i=0}^{p} g_i(\boldsymbol{S}) \wedge t = \sum_{\boldsymbol{b}\in\{0,1\}^\mu} g(\boldsymbol{b})$$

Our scheme, dubbed $\mathsf{CP}_{\mathsf{sc}}$, is built as a generalization of the protocol recently proposed in [66, 68] that works for a relation that is the same as the above one except that only t is committed while g is public to the verifier. For the reader familiar with the zero-knowledge sum-check protocol in [68, Construction 2], what we do here is to modify their protocol using the following ideas: whereas in [68] the verifier has access to g and computes a commitment to $g(\boldsymbol{s})$ for a random point $\boldsymbol{s}$ on its own, in our case the verifier has access to a commitment c_g of g and we let the prover create a commitment to $g(\boldsymbol{s})$ and use $\mathsf{CP}_{\mathsf{poly}}$ to prove its correctness with respect to c_g. More precisely, the verifier does not have a commitment to g but rather commitments to the factors of g. Hence our prover proceeds by additionally creating commitments to each $g_i(\boldsymbol{s})$, it proves their correct evaluations and then uses $\mathsf{CP}_{\mathsf{prd}}$ to prove that $g(\boldsymbol{s}) = \prod_{i=0}^{p} g_i(\boldsymbol{s})$ with respect to these commitments. Making these changes results in a protocol that is the same as that in [68] except for the last round from the prover to the verifier. Indeed we can prove the security of our protocol by making a reduction to the one of [68]. We state the following result. The full version

contains its proof and a detailed description of this protocol for the case $p = 2$, which is sufficient for our applications.

Efficiency. In $\mathsf{CP}_{\mathsf{sc}}$, the verifier needs time $O(\mu)$ plus the time to compute $g_0(\boldsymbol{s})$. The prover's costs include the running time in the sum-check protocol and the creation of the $\mathsf{CP}_{\mathsf{poly}}$ proofs. If the g_i are multilinear, $\mathsf{CP}_{\mathsf{poly}}.\mathsf{Prove}$'s time is $O(2^\mu)$. Also, from [60], if the polynomials g_i allow for evaluation in $O(\mu)$ time or are MLE of vectors, the prover's cost in sum-check can be reduced to $O(2^\mu)$.

Theorem 5.2. *Assume* PolyCom *is an extractable linearly homomorphic commitment,* $\mathsf{CP}_{\mathsf{poly}}$ *and* $\mathsf{CP}_{\mathsf{prd}}$ *are zkSNARKs for relations* R^{poly} *and* R^{prd} *respectively, and Construction 2 in [68] is a ZK interactive argument for sum-check. Then there is a ZK interactive argument for relation* R^{sc}*. Furthermore, by applying the Fiat-Shamir heuristic we get a zkSNARK in the random oracle model, that we call* $\mathsf{CP}_{\mathsf{sc}}$*.*

5.3 A CP-SNARK for Hadamard Products

In this section we propose a CP-SNARK for PolyCom for the relation R^{had} over $(\mathbb{F}^m)^3$ such that

$$R^{\mathsf{had}}(\boldsymbol{u}_0, \boldsymbol{u}_1, \boldsymbol{u}_2) = 1 \iff \forall i \in [m] : u_{0,i} = u_{1,i} \cdot u_{2,i}$$

Let $m = 2^\mu$ and let $\tilde{u}_j : \mathbb{F}^\mu \to \mathbb{F}$ be the MLE of $\boldsymbol{u}_j$. Clearly, the relation holds iff for all $\boldsymbol{b} \in \{0,1\}^\mu$ we have $\tilde{u}_0(\boldsymbol{b}) = \tilde{u}_1(\boldsymbol{b}) \cdot \tilde{u}_2(\boldsymbol{b})$. If the relation holds, observe that the polynomial $\tilde{u}_1(\boldsymbol{X}) \cdot \tilde{u}_2(\boldsymbol{X})$ is an extension of the vector $\boldsymbol{u}_0$, but not a multilinear one. From Lemma 5.1 the following equality holds

$$\tilde{u}_0(\boldsymbol{X}) = \sum_{\boldsymbol{b} \in \{0,1\}^\mu} \tilde{eq}(\boldsymbol{X}, \boldsymbol{b}) \cdot \tilde{u}_1(\boldsymbol{b}) \cdot \tilde{u}_2(\boldsymbol{b})$$

Without considering zero-knowledge, the main idea of our protocol is that, to check the above equality, the verifier starts by picking a random point $\boldsymbol{r} \leftarrow\$ \mathbb{F}^\mu$, and then the prover uses $\mathsf{CP}_{\mathsf{sc}}$ to show that $t = \tilde{u}_0(\boldsymbol{r}) = \sum_{\boldsymbol{b} \in \{0,1\}^\mu} g(\boldsymbol{b})$, where $g(\boldsymbol{S}) = \tilde{eq}(\boldsymbol{r}, \boldsymbol{S}) \cdot \tilde{u}_1(\boldsymbol{S}) \cdot \tilde{u}_2(\boldsymbol{S})$. Notice indeed that g can be written as the product of three polynomials $g(\boldsymbol{S}) := \prod_0^2 g_i(\boldsymbol{S})$, of which the first one is public: $g_1(\boldsymbol{S}) = \tilde{u}_1(\boldsymbol{S})$, $g_2(\boldsymbol{S}) = \tilde{u}_2(\boldsymbol{S})$ and $g_0(\boldsymbol{S}) := \tilde{eq}(\boldsymbol{r}, \boldsymbol{S})$. Finally, the prover also needs to convince the verifier that $t = \tilde{u}_0(\boldsymbol{r})$, which is done using a CP-SNARK $\mathsf{CP}_{\mathsf{poly}}$ for proving correctness of polynomial evaluations.

Therefore we build a CP-SNARK $\mathsf{CP}_{\mathsf{had}}$ for R^{had} and PolyCom by using CP-SNARKs $\mathsf{CP}_{\mathsf{poly}}, \mathsf{CP}_{\mathsf{sc}}$ for PolyCom as building blocks. Furthermore, we describe the scheme as a non-interactive one by letting $\boldsymbol{r} \leftarrow H((c_j)_{j \in [:3]})$ using the random oracle model for H.

The full description of the scheme is given below.

$\mathsf{KeyGen}(\mathsf{ck}) \to (\mathsf{ek} := (\mathsf{ck}, \mathsf{ek}_s, \mathsf{ek}_p, H), \mathsf{vk} := (\mathsf{cvk}, \mathsf{vk}_s, \mathsf{vk}_p, H))$

$(\mathsf{ek}_s, \mathsf{vk}_s) \leftarrow \mathsf{CP}_{\mathsf{sc}}.\mathsf{KeyGen}(\mathsf{ck})$; $(\mathsf{ek}_p, \mathsf{vk}_p) \leftarrow \mathsf{CP}_{\mathsf{poly}}.\mathsf{KeyGen}(\mathsf{ck})$

$\mathsf{Prove}\big(\mathsf{ek}, (c_j)_{j \in [:3]}, (\boldsymbol{u}_j)_{j \in [:3]}, (o_j)_{j \in [:3]}\big) \to \pi := (c_t, \pi_0, \pi_{\mathsf{sc}})$

$\boldsymbol{r} \leftarrow H((c_j)_{j \in [:3]})$; $t \leftarrow \tilde{u}_0(\boldsymbol{r})$; $(c_t, o_t) \leftarrow \mathsf{ComVal}(\mathsf{ck}, t)$

$\pi_0 \leftarrow \mathsf{CP}_{\mathsf{poly}}.\mathsf{Prove}(\mathsf{ek}_p, \boldsymbol{r}, (c_0, c_t), (\tilde{u}_0, t), (o_0, o_t))$

$\pi_{\mathsf{sc}} \leftarrow \mathsf{CP}_{\mathsf{sc}}.\mathsf{Prove}(\mathsf{ek}_s, \widetilde{eq}(\boldsymbol{r}, \boldsymbol{S}), (c_t, c_1, c_2), (t, o_t, \tilde{u}_1, o_1, \tilde{u}_2, o_2))$

$\mathsf{VerProof}\big(\mathsf{vk}, (c_j)_{j \in [:3]}, \pi\big) \to b$

$\boldsymbol{r} \leftarrow H((c_j)_{j \in [:3]})$; $b \leftarrow \mathsf{CP}_{\mathsf{poly}}.\mathsf{VerProof}(\mathsf{vk}_p, \boldsymbol{r}, c_0, c_t, \pi_0)$

$b \leftarrow b \wedge \mathsf{CP}_{\mathsf{sc}}.\mathsf{VerProof}(\mathsf{vk}_s, \widetilde{eq}(\boldsymbol{r}, \boldsymbol{S}), (c_t, c_1, c_2), \pi_{\mathsf{sc}})$

Efficiency. Computing π_0 takes time $O(m)$, and the same holds for π_{sc}. The latter follows by observing that the factors of $g(\boldsymbol{S})$ satisfy the good efficiency conditions for $\mathsf{CP}_{\mathsf{sc}}$, i.e., $\widetilde{eq}(\boldsymbol{r}, \boldsymbol{s})$ can be computed in $O(\mu)$ time and $\tilde{u}_1, \tilde{u}_2$ are MLE of vectors of length $m = 2^\mu$. For similar reasons, the verifier's time is $O(\mu)$.

We state the following result:

Theorem 5.3. *In the random oracle model, assuming that* PolyCom *is an extractable trapdoor commitment,* $\mathsf{CP}_{\mathsf{poly}}, \mathsf{CP}_{\mathsf{sc}}$ *are zero-knowledge CP-SNARKs for* PolyCom *and relations* R^{poly} *and* R^{sc} *respectively, then the scheme* $\mathsf{CP}_{\mathsf{had}}$ *described above is a zero-knowledge CP-SNARK for* PolyCom *and relation* R^{had}*.*

5.4 A CP-SNARK for Self Permutation

In this section we propose a CP-SNARK for PolyCom for the self-permutation relation R_ϕ^{sfprm} such that $R_\phi^{\mathsf{sfprm}}(\boldsymbol{y} := (\boldsymbol{x}, (\boldsymbol{u}_j)_{j \in [\ell]})) = 1$ $\iff \forall i \in [m] : y_i = y_{\phi(i)}$, where $\phi : [m] \to [m]$ is a permutation.

Our scheme uses a probabilistic test to prove a permutation of vectors due to [17, 37]. For this we need of a CP-SNARK for proving that $t = \prod_{i=1}^m y_i$ with respect to a commitment to point t and vector $\boldsymbol{y}$. We call such a relation *internal product* R^{ipd}.

In what follows we present the main ideas to build a CP-SNARK for R^{sfprm} from one for R^{ipd}. Next, we discuss how a CP-SNARK for internal products can be instantiated.

Recall that the goal is to prove that, for a permutation $\phi : [m] \to [m]$ a committed vector $\boldsymbol{y}$ satisfies $y_i = y_{\phi(i)}, \forall i \in [m]$. Consider the following vectors in $\mathbb{F}^m$, $\mathbf{1}$, $\boldsymbol{v} = (1, \ldots, m)$, and $\boldsymbol{\phi} = (\phi(1), \ldots, \phi(m))$, and assume that the prover committed to $\boldsymbol{y}$. Let the verifier choose two random values $r, s \leftarrow\$ \mathbb{F}$ and define the vectors $\boldsymbol{y}' := \boldsymbol{y} + r \cdot \boldsymbol{v} - s \cdot \mathbf{1}$ and $\boldsymbol{y}'' := \boldsymbol{y} + r \cdot \boldsymbol{\phi} - s \cdot \mathbf{1}$.

If $\boldsymbol{y}$ is a permutation of itself according to ϕ, then $(\boldsymbol{y} + r \cdot \boldsymbol{\phi})$ is a permutation of $(\boldsymbol{y} + r \cdot \boldsymbol{v})$ according to ϕ; however, if $\boldsymbol{y}$ is *not* a self-permutation according to ϕ then with overwhelming probability over the choice of r some of the entries of $\boldsymbol{y} + r \cdot \boldsymbol{\phi}$ will not be in the vector $\boldsymbol{y} + r \cdot \boldsymbol{v}$. In our scheme the idea is to let the prover show that $\prod_i y'_i = z = \prod_i y''_i$ using $\mathsf{CP}_{\mathsf{ipd}}$ on $(z, \boldsymbol{y}')$ and $(z, \boldsymbol{y}'')$. However, if some entries of $\boldsymbol{y} + r\cdot$ are not in $\phi \neq \boldsymbol{y} + r \cdot \boldsymbol{v}$, $\prod_i (y_i + r \cdot i - s) = \prod_i (y_i + r \cdot \phi(i) - s)$ holds with negligible probability over the choice of s by the Schwartz-Zippel lemma, thus a prover can be succesful only by cheating with $\mathsf{CP}_{\mathsf{ipd}}$.

We consider an instantiation of $\mathsf{CP}_{\mathsf{ipd}}$ based on Thaler's protocol for trees of multiplications [60]. R^{ipd} can be expressed with an arithmetic circuit that is a tree of multiplications over $m = 2^\mu$ inputs. Thaler showed that for this specially regular circuit the CMT protocol can be adapted so that the prover and verifier run in time $O(m)$ and $O(\mu^2)$, respectively. To build a CP-SNARK for R^{ipd}, we thus modify the zk-vSQL protocol [68, Construction 3] so as to work over Thaler's protocol instead of CMT. The changes are quite minimal and mainly regard the equation that links the adjacent layers of the tree. We show this protocol in the full version.

One detail to be noted here is that such $\mathsf{CP}_{\mathsf{ipd}}$ works with binary tree circuits, meaning that their input should be a power of two length, so we tweak our definition of the self-permutation relation accordingly. We must work on $\ell + 1$ vectors such that, each has length $n_j = 2^{\mu_j}$ (this is immediate since we commit to MLEs of vectors) but their concatenation has length $\sum_{j=0}^{\ell} n_j = m$ which may not be a power-of-two.

To solve this issue, we execute $\mathsf{CP}_{\mathsf{ipd}}$ on each block and then aggregate the $\ell+1$ committed results using a simple zero-knowledge argument for proving a product relation over three commitments, i.e., $\mathsf{CP}_{\mathsf{prd}}$. This results in about $\ell + 1$ calls to $\mathsf{CP}_{\mathsf{ipd}}$ and $\mathsf{CP}_{\mathsf{prd}}$. Although this makes proofs grow with ℓ, we observe that in all our applications ℓ is some small constant, e.g., $8 - 10$ in our arithmetic circuits encoding.

Efficiency. From the efficiency observations about $\mathsf{CP}_{\mathsf{ipd}}$ given above, we get that $\mathsf{CP}_{\mathsf{sfprm}}.\mathsf{Prove}$ and $\mathsf{CP}_{\mathsf{sfprm}}.\mathsf{VerProof}$ run in time $O(m)$ and $O(\log^2 m)$ respectively.

Theorem 5.4. *In the random oracle model, assuming that* PolyCom *is an extractable and linearly-homomorphic trapdoor commitment,* $\mathsf{CP}_{\mathsf{ipd}}$, $\mathsf{CP}_{\mathsf{prd}}$ *are zero-knowledge CP-SNARKs for* PolyCom *and relations* R^{ipd} *and* R^{prd} *respectively, then* $\mathsf{CP}_{\mathsf{sfprm}}$ *in Figure 3 is a zero-knowledge CP-SNARK for* PolyCom *and relation* R^{sfprm}.

5.5 A CP-SNARK for Matrix Multiplication

In this section we propose a CP-SNARK for PolyCom for the relation R^{mm} over $\mathcal{D}_A \times \mathcal{D}_B \times \mathcal{D}_C$ where $\mathcal{D}_A = \mathbb{F}^{n\times n'}$, $\mathcal{D}_B = \mathbb{F}^{n'\times m}$, $\mathcal{D}_C = \mathbb{F}^{n\times m}$ and $R^{\mathsf{mm}}(\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C}) = 1 \iff \boldsymbol{C} = \boldsymbol{A} \cdot \boldsymbol{B}$. Namely, for two committed matrices $\boldsymbol{A}$ and $\boldsymbol{B}$, one can prove that another committed matrix $\boldsymbol{C}$ equals to their product.

The scheme is inspired by the interactive proof of matrix multiplication of Thaler [60], making it a ZK argument with similar ideas to those in [68]. We build our scheme for the polynomial commitment scheme PolyCom and CP-SNARKs $\mathsf{CP}_{\mathsf{poly}}$ and $\mathsf{CP}_{\mathsf{sc}}$ for the relations R^{poly} and R^{sc} for factored polynomials.

We present this scheme for square matrices for readability, but the protocol can be simply adapted to the general form. Let matrices $\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C} \in \mathbb{F}^{n\times n}$, we build their multilinear extension $\tilde{C} \in \mathbb{F}^{\mu} \times \mathbb{F}^{\mu} \to \mathbb{F}$ with $\mu = \log n$, which is the unique polynomial such that $\tilde{C}(i_1, \ldots, i_\mu, j_1, \ldots, j_\mu) = C_{i,j}$ if $\{i_k\}_{k=1}^{\mu}$ and $\{j_k\}_{k=1}^{\mu}$ are the binary representation of indexes i and j (resp. $\tilde{A}$ and $\tilde{B}$).

Then, we can represent matrix multiplication as

$$\tilde{C}(\boldsymbol{I}, \boldsymbol{J}) = \sum\nolimits_{\boldsymbol{b}\in\{0,1\}^{\mu}} \tilde{A}(\boldsymbol{I}, \boldsymbol{b}) \cdot \tilde{B}(\boldsymbol{b}, \boldsymbol{J})$$

where $(\boldsymbol{I}, \boldsymbol{J})$ could be seen as as a single vector of length 2μ. However, we will stick to this notation instead as it makes clearer that multilinearity is conserved after the product of $\tilde{A}$ and $\tilde{B}$.

The protocol works as follows. The prover evaluates $\tilde{C}$ on a randomly chosen value $(\boldsymbol{\rho}|\boldsymbol{\sigma})$ and obtains a proof that the output t is indeed the result of the polynomial evaluation $t = \tilde{C}(\boldsymbol{\rho}, \boldsymbol{\sigma})$ using $\mathsf{CP}_{\mathsf{poly}}$. Then, the prover convinces the verifier that $\tilde{C}$ is well-formed using $\mathsf{CP}_{\mathsf{sc}}$, i.e. $t = \sum_{\boldsymbol{b}\in\{0,1\}^{\mu}} g(b)$. Here $g(\boldsymbol{S}) = \prod_0^2 g_i(\boldsymbol{S})$, where $g_1(\boldsymbol{S}) := \tilde{A}(\rho, \boldsymbol{S})$, $g_2(\boldsymbol{S}) := \tilde{B}(\boldsymbol{S}, \sigma)$ and $g_0(\boldsymbol{S}) := 1$ is the all-ones constant polynomial.

Efficiency. The cost of this scheme is given by the complexity of $\mathsf{CP}_{\mathsf{poly}}$ and $\mathsf{CP}_{\mathsf{sc}}$. The proving time of the former is linear in the number of monomials of the polynomial $\tilde{C}$, which is $2^{2\mu}$ by construction. Similarly, the latter's is linear in the monomials of $g(\boldsymbol{S})$, which is again $2^{2\mu}$. This makes a linear prover in the number of elements ($N = n^2$). The verifier runtime is linear in the number of variables of the polynomials (i.e. 2μ).

$\mathsf{CP}_{\mathsf{sfprm}}.\mathsf{KeyGen}(\mathsf{ck}) \to (\mathsf{ek} := (\mathsf{ck}, \mathsf{ek}_p), \mathsf{vk} := (\mathsf{cvk}, \mathsf{vk}_p)) :$

$(\mathsf{ek}_p, \mathsf{vk}_p) \leftarrow \mathsf{CP}_{\mathsf{ipd}}.\mathsf{KeyGen}(\mathsf{ck})$

$\mathsf{CP}_{\mathsf{sfprm}}.\mathsf{Derive}((\mathsf{ek}, \mathsf{vk}), R_{\phi}^{\mathsf{sfprm}}) \to (\mathsf{ek}_{\phi}, \mathsf{vk}_{\phi}) :$

for $j = 0, \ldots, \ell$:

$(c_{1,j}, o_{1,j}) \leftarrow \mathsf{ComPoly}^*(\mathsf{ck}, \tilde{1}_j)$

$(c_{v,j}, o_{v,j}) \leftarrow \mathsf{ComPoly}^*(\mathsf{ck}, \tilde{v}_j)$

$(c_{\phi,j}, o_{\phi,j}) \leftarrow \mathsf{ComPoly}^*(\mathsf{ck}, \tilde{\phi}_j)$

$\mathsf{ek}_{\phi} := (\mathsf{ek}, \{c_{1,j}, o_{1,j}, c_{v,j}, o_{v,j}, c_{\phi,j}, o_{\phi,j}\}_{j=0}^{\ell}, \phi)$

$\mathsf{vk}_{\phi} := (\mathsf{vk}, \{c_{1,j}, c_{v,j}, c_{\phi,j}\}_{j=0}^{\ell})$

$\mathsf{CP}_{\mathsf{sfprm}}.\mathsf{Prove}^*(\mathsf{ek}_{\phi}, \boldsymbol{x}, (c_j)_{j\in[\ell]}, (\boldsymbol{u}_j)_{j\in[\ell]}, (o_j)_{j\in[\ell]}) \to \pi :$

$(r, s) \leftarrow H((c_{\phi,j})_{j\in[0,\ell]}, \boldsymbol{x}, (c_j)_{j\in[\ell]})$ and let $\boldsymbol{\rho} = (1, r, -s)$

$(c_0, o_0) \leftarrow \mathsf{ComPoly}(\mathsf{ck}, \tilde{x})$

for $j = 0, \ldots, \ell$:

$(c'_j, o'_j) \leftarrow \mathsf{HomEval}(\mathsf{ck}, \boldsymbol{\rho}, (c_j, c_{v,j}, c_{1,j}), (o_j, o_{v,j}, o_{1,j}))$

$(c''_j, o''_j) \leftarrow \mathsf{HomEval}(\mathsf{ck}, \boldsymbol{\rho}, (c_j, c_{\phi,j}, c_{1,j}), (o_j, o_{\phi,j}, o_{1,j}))$

$\boldsymbol{y}'_j := \boldsymbol{y}_j + r \cdot \boldsymbol{v}_j - s \cdot \boldsymbol{1}_j \quad ; \quad z'_j := \prod_{i=1}^{n_j} y'_{j,i}$

$\boldsymbol{y}''_j := \boldsymbol{y}_j + r \cdot \boldsymbol{\phi}_j - s \cdot \boldsymbol{1}_j \quad ; \quad z''_j := \prod_{i=1}^{n_j} y''_{j,i}$

$(c_{z'_j}, o_{z'_j}) \leftarrow \mathsf{ComVal}(\mathsf{ck}, z'_j)$; $(c_{z''_j}, o_{z''_j}) \leftarrow \mathsf{ComVal}(\mathsf{ck}, z''_j)$

$\pi'_j \leftarrow \mathsf{CP}_{\mathsf{ipd}}.\mathsf{Prove}\big(\mathsf{ek}_p, c_{z'_j}, (c'_{j,i})_{i\in[n_j]}, z'_j, (y'_{j,i})_{i\in[n_j]}, o_{z'_j}, (o'_{j,i})_{i\in[n_j]}\big)$

$\pi''_j \leftarrow \mathsf{CP}_{\mathsf{ipd}}.\mathsf{Prove}\big(\mathsf{ek}_p, c_{z''_j}, (c''_{j,i})_{i\in[n_j]}, z''_j, (y''_{j,i})_{i\in[n_j]}, o_{z''_j}, (o''_{j,i})_{i\in[n_j]}\big)$

if $j \stackrel{?}{=} 0 : w'_0 := z'_0$; $w''_0 := z''_0$; $c_{w'_0} := c_{z'_0}$; $c_{w''_0} := c_{z''_0}$; **else** :

$w'_j \leftarrow w'_{j-1} \cdot z'_j$; $(c_{w'_j}, o_{w'_j}) \leftarrow \mathsf{ComVal}(\mathsf{ck}, w'_j)$

$w''_j \leftarrow w''_{j-1} \cdot z''_j$; $(c_{w''_j}, o_{w''_j}) \leftarrow \mathsf{ComVal}(\mathsf{ck}, w''_j)$

$\pi_{w'_j} \leftarrow \mathsf{CP}_{\mathsf{prd}}.\mathsf{Prove}(\mathsf{ck}, c_{w'_{j-1}}, c_{z'_j}, c_{w'_j}, w'_{j-1}, z'_j, w'_j, o_{w'_{j-1}}, o_{z'_j}, o_{w'_j})$

$\pi_{w''_j} \leftarrow \mathsf{CP}_{\mathsf{prd}}.\mathsf{Prove}(\mathsf{ck}, c_{w''_{j-1}}, c_{z''_j}, c_{w''_j}, w''_{j-1}, z''_j, w''_j, o_{w''_{j-1}}, o_{z''_j}, o_{w''_j})$

endif ; **endfor**

$\pi_z \leftarrow \mathsf{CP}_{\mathsf{eq}}.\mathsf{Prove}(\mathsf{ck}, c_{w'_\ell}, c_{w''_\ell}, w'_\ell, w''_\ell, o_{w'_\ell}, o_{w''_\ell})$

return $\pi := (c_0, o_0, \{c_{z'_j}, c_{z''_j}, c_{w'_j}, c_{w''_j}, \pi'_j, \pi''_j\}_{j=0}^{\ell}, \{\pi_{w'_j}, \pi_{w''_j}\}_{j=1}^{\ell}, \pi_z)$

$\mathsf{CP}_{\mathsf{sfprm}}.\mathsf{VerProof}^*\big(\mathsf{vk}_{\phi}, \boldsymbol{x}, (c_j)_{j\in[\ell]}, \pi\big) \to b :$

$(r, s) \leftarrow H((c_{\phi,j})_{j\in[0,\ell]}, \boldsymbol{x}, (c_j)_{j\in[\ell]})$ and let $\boldsymbol{\rho} = (1, r, -s)$

$b \leftarrow \mathsf{VerCommit}(\mathsf{cvk}, c_0, \tilde{x}, o_0)$

for $j = 0, \ldots, \ell$:

$(c'_j, \cdot) \leftarrow \mathsf{HomEval}(\mathsf{cvk}, \boldsymbol{\rho}, (c_j, c_{v,j}, c_{1,j}), \cdot)$

$(c''_j, \cdot) \leftarrow \mathsf{HomEval}(\mathsf{cvk}, \boldsymbol{\rho}, (c_j, c_{\phi,j}, c_{1,j}), \cdot)$

$b \leftarrow b \wedge \mathsf{CheckCom}(\mathsf{cvk}, c_{z'_j}) \wedge \mathsf{CP}_{\mathsf{ipd}}.\mathsf{VerProof}(\mathsf{vk}_p, c_{z'_j}, (c'_{j,i})_{i\in[n_j]})$

$\wedge\ \mathsf{CheckCom}(\mathsf{cvk}, c_{z''_j}) \wedge \mathsf{CP}_{\mathsf{ipd}}.\mathsf{VerProof}(\mathsf{vk}_p, c_{z''_j}, (c''_{j,i})_{i\in[n_j]})$

$\wedge\ \mathsf{CheckCom}(\mathsf{cvk}, c_{w'_j}) \wedge \mathsf{CheckCom}(\mathsf{cvk}, c_{w''_j})$

if $j \neq 0 : b \leftarrow b \wedge \mathsf{CP}_{\mathsf{prd}}.\mathsf{VerProof}(\mathsf{cvk}, c_{w'_{j-1}}, c_{z'_j}, c_{w'_j}, \pi_{w'_j})$

$\wedge\ \mathsf{CP}_{\mathsf{prd}}.\mathsf{VerProof}(\mathsf{cvk}, c_{w''_{j-1}}, c_{z''_j}, c_{w''_j}, \pi_{w''_j})$

endif ; **endfor**

$b \leftarrow b \wedge \mathsf{CP}_{\mathsf{eq}}.\mathsf{VerProof}(\mathsf{cvk}, c_{w'_\ell}, c_{w''_\ell}, \pi_z)$

Figure 3: CP-SNARK for specializable universal R^{sfprm}

$\mathsf{CP}_{\mathsf{mm}}.\mathsf{KeyGen}(\mathsf{ck}) \to (\mathsf{ek}, \mathsf{vk})$:

$(\mathsf{ek}_s, \mathsf{vk}_s) \leftarrow \mathsf{CP}_{\mathsf{sc}}.\mathsf{KeyGen}(ck)$; $(\mathsf{ek}_p, \mathsf{vk}_p) \leftarrow \mathsf{CP}_{\mathsf{poly}}.\mathsf{KeyGen}(ck)$

return $(\mathsf{ek} := (\mathsf{ck}, \mathsf{ek}_p, \mathsf{ek}_s, H), \mathsf{vk} := (\mathsf{cvk}, \mathsf{vk}_p, \mathsf{vk}_s, H))$

$\mathsf{CP}_{\mathsf{mm}}.\mathsf{Prove}(\mathsf{ek}, c_A, c_B, c_C, \boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C}, o_A, o_B, o_C,) \to \pi$:

$(\boldsymbol{\rho}|\boldsymbol{\sigma}) \leftarrow H(c_A, c_B, c_C)$; $t \leftarrow \tilde{C}(\boldsymbol{\rho}, \boldsymbol{\sigma})$; $(c_t, o_t) \leftarrow \mathsf{ComVal}(\mathsf{ck}, t)$

Let $g(S) := \tilde{A}(\boldsymbol{\rho}, S) \cdot \tilde{B}(S, \boldsymbol{\sigma}) \equiv g_1(S) \cdot g_2(S)$

$\pi_t \leftarrow \mathsf{CP}_{\mathsf{poly}}.\mathsf{Prove}(\mathsf{ek}_p, (\boldsymbol{\rho}|\boldsymbol{\sigma}), (c_C, c_t), (\tilde{C}, t), (o_C, o_t))$

$\pi_{\mathsf{sc}} \leftarrow \mathsf{CP}_{\mathsf{sc}}.\mathsf{Prove}(\mathsf{ek}_s, g_0(S) := 1, (c_t, c_A, c_B), (t, o_t, \tilde{A}, o_A, \tilde{B}, o_B))$

return $\pi \leftarrow (c_t, \pi_t, \pi_{\mathsf{sc}})$

$\mathsf{CP}_{\mathsf{mm}}.\mathsf{VerProof}(\mathsf{vk}, c_A, c_B, c_C, \pi) \to b \in \{0, 1\}$:

$(\boldsymbol{\rho}, \boldsymbol{\sigma}) \leftarrow H(c_A, c_B, c_C)$

$b \leftarrow \mathsf{CP}_{\mathsf{poly}}.\mathsf{VerProof}(\mathsf{vk}_p, (\boldsymbol{\rho}|\boldsymbol{\sigma}), c_C, c_t, \pi_t)$

$\wedge\ \mathsf{CP}_{\mathsf{sc}}.\mathsf{VerProof}(\mathsf{vk}_s, g_0(S) := 1, (c_t, c_A, c_B), \pi_{\mathsf{sc}})$

Theorem 5.5. *In the random oracle model, assuming that* PolyCom *is an extractable trapdoor commitment,* $\mathsf{CP}_{\mathsf{poly}}$ *and* $\mathsf{CP}_{\mathsf{sc}}$ *are zk-SNARKs for* PolyCom *and relations* R^{poly} *and* R^{sc} *respectively, then the scheme* $\mathsf{CP}_{\mathsf{mm}}$ *described above is a zkSNARK for* PolyCom *and relation* R^{mm}.

In the full version, we provide a CP-SNARK for matrix multiplication where the output matrix is known. This version is more efficient than the obvious solution of opening the commitment to $\boldsymbol{C}$ in the present scheme. We also provide a very similar scheme, $\mathsf{CP}_{\mathsf{lin}}$, for linear properties of committed vectors.

6 LEGOSNARK APPLICATIONS AND EVALUATION

In this section we show how to use the modular commit-and-prove approach to obtain new CP-SNARKs for computation expressible by arithmetic circuits (ACs) and we discuss the resulting instantiations. First, the following preliminar relation must be presented:

Equalities Among Vector Entries. A common building block in both schemes of this section is a system for proving that the entries of a vector satisfy a set of equalities between them. Namely, given a set S of pairs of indices (i, k), we define a relation R_S^{veq} that holds for a vector $\boldsymbol{u}$ iff $u_i = u_k$ for all $(i, k) \in S$. In what follows we discuss different ways to encode this relation:

Definition 6.1 (Relation for equalities among vector entries). Let $\mathcal{D}$ be some domain (e.g., a finite field $\mathbb{F}$), let $n_0, \ldots, n_\ell$ be positive integers such that $\mathcal{D}_j := \mathcal{D}^{n_j}$ and let $m = \sum_{j=0}^{\ell} n_j$. Given a set $S = \{(i_1, k_1), \ldots, (i_l, k_l)\} \subset [m] \times [m]$, we define a relation R_S^{veq} over $\mathcal{D}_0 \times \cdots \times \mathcal{D}_\ell = \mathcal{D}^m$ such that:

$R_S^{\mathsf{veq}}(\boldsymbol{y} := (\boldsymbol{x}, (\boldsymbol{u}_j)_{j\in[\ell]})) = 1 \iff \forall (i, k) \in S : y_i = y_k$.

In what follows we discuss different ways to encode the above relation.

Relation R_S^{veq} can be expressed using R_ϕ^{sfprm} (see Section 5.4) for an appropriate permutation ϕ that encodes S. The idea is that a set $S \subset [m] \times [m]$ can be seen as the description of an undirected graph with $2m$ vertices. From S it is possible to extract another set $S' \subset [m] \times [m]$ that contains a cycle $((i_1, k_1), \ldots, (i_t, k_t))$ for every connected component of the graph represented by S. Taking the product of all the cycles in S' defines a permutation $\phi : [m] \to [m]$ such that $\forall (i, k) \in S : y_i = y_k$ iff $\forall j \in [m] : y_j = y_{\phi(j)}$. Then for such ϕ computed from S we have $R_S^{\mathsf{veq}}(\boldsymbol{x}, (\boldsymbol{u}_j)_{j\in[\ell]}) \iff R_\phi^{\mathsf{sfprm}}(\boldsymbol{x}, (\boldsymbol{u}_j)_{j\in[\ell]})$.

At this point one can either design a proof system for R_ϕ^{sfprm} (as in Section 5.4) or use an alternative encoding of R_ϕ^{sfprm} based on linear constraints. The idea is to evaluate the relation on a vector $\boldsymbol{y} \in \mathbb{F}^m$ by checking that $\boldsymbol{Z} \cdot \boldsymbol{y} = \boldsymbol{0}$, where $\boldsymbol{Z} \in \mathbb{F}^{m' \times m}$, with $m' \leq m$, is the matrix obtained by removing the zero rows from $(\boldsymbol{I} - \Sigma_\phi) \in \mathbb{F}^{m \times m}$, where Σ_ϕ is the permutation matrix representing ϕ. Then clearly $R_\phi^{\mathsf{sfprm}}(\boldsymbol{x}, (\boldsymbol{u}_j)_{j\in[\ell]})$ holds iff $R_{\boldsymbol{Z}}^{\mathsf{lin}}(\boldsymbol{0}, \boldsymbol{x}, (\boldsymbol{u}_j)_{j\in[\ell]})$ holds, where the relation R^{lin}, modelling the linear property over (committed) vectors, is formally defined as follows.

Definition 6.2 (Linear property relation). Let $n_1, n_2, m_1, \ldots, m_\ell$ be integers such that $\{\mathcal{D}_{x,j} := \mathcal{D}^{n_j}\}_{j\in[1,2]}$, $\{\mathcal{D}_{u,j} := \mathcal{D}^{m_j}\}_{j\in[\ell]}$, and $m = n_2 + \sum_{j=1}^{\ell} m_j$. Given a matrix $\boldsymbol{F} \in \mathcal{D}^{n_1 \times m}$, we define a relation $R_{\boldsymbol{F}}^{\mathsf{lin}}$ over $\mathcal{D}_{x,1} \times \mathcal{D}_{x,2} \times \mathcal{D}_{u,1} \times \cdots \times \mathcal{D}_{u,\ell}$ such that:

$$R_{\boldsymbol{F}}^{\mathsf{lin}}(\boldsymbol{x}_1, \boldsymbol{x}_2, (\boldsymbol{u}_j)_{j\in[\ell]}) = 1 \iff \boldsymbol{F} \cdot \boldsymbol{y} = \boldsymbol{x}_1, \text{ where } \boldsymbol{y} := (\boldsymbol{x}_2, (\boldsymbol{u}_j)_{j\in[\ell]})$$

Note that the above relation R^{lin} is slightly different from the one supported by $\mathsf{CP}_{\mathsf{lin}}^{\mathsf{Ped}}$ of Section 4.2. The only difference is that in $\mathsf{CP}_{\mathsf{lin}}^{\mathsf{Ped}}$ the linear function is not applied over public inputs. However, this small discrepancy can be easily solved by adding a commitment to the additional public input $\boldsymbol{x}_2$ and opening this commitment.

CP-SNARKs for Arithmetic Circuits. Bootle et al. [16, 20] show that satisfiability of an arithmetic circuit C holds if the assignment of the inputs-output wires of multiplication gates satify an Hadamard product relation and a set of linear constraints, i.e., if there are three vectors $\boldsymbol{u}_L^M, \boldsymbol{u}_R^M, \boldsymbol{u}_O^M \in \mathbb{F}^N$ such that

$$\boldsymbol{u}_L^M \circ \boldsymbol{u}_R^M = \boldsymbol{u}_O^M \ \wedge\ \boldsymbol{W}_L \cdot \boldsymbol{u}_L^M + \boldsymbol{W}_R \cdot \boldsymbol{u}_R^M + \boldsymbol{W}_O \cdot \boldsymbol{u}_O^M = \boldsymbol{c} \quad (3)$$

where $\boldsymbol{W}_L, \boldsymbol{W}_R, \boldsymbol{W}_O \in \mathbb{F}^{Q \times N}, \boldsymbol{c} \in \mathbb{F}^Q$ can be extracted from C's description. By defining $\boldsymbol{F} := [\boldsymbol{W}_L | \boldsymbol{W}_R | \boldsymbol{W}_O]$, we can encode (3) as $R^{\mathsf{had}}(\boldsymbol{u}_L^M, \boldsymbol{u}_R^M, \boldsymbol{u}_O^M) \wedge R_{\boldsymbol{F}}^{\mathsf{lin}}(\boldsymbol{c}, (\boldsymbol{u}_L^M, \boldsymbol{u}_R^M, \boldsymbol{u}_O^M))$. Hence, by applying our Theorem 3.2, we obtain a CP-SNARK for AC, dubbed LegoAC.

Instantiations. We evaluate two instantiations of LegoAC:

- LegoAC1: from our $\mathsf{CP}_{\mathsf{lin}}^{\mathsf{Ped}}$ (Section 4.2) and Lipmaa's CP-SNARK for Hadamard products [49]. LegoAC1 is a CP-SNARK for the commitment scheme of [49], and its security holds in the generic group model (due to GGM security of $\mathsf{CP}_{\mathsf{lin}}^{\mathsf{Ped}}$).
- LegoAC2: from our $\mathsf{CP}_{\mathsf{lin}}^{\mathsf{Ped}}$ (Section 4.2) and our $\mathsf{CP}_{\mathsf{had}}$ (Section 5.3). This is a CP-SNARK for PolyCom, and its security holds in the GGM and random oracle model (the latter due to $\mathsf{CP}_{\mathsf{had}}$).

If needed, both schemes can be lifted to work with a standard Pedersen commitment using $\mathsf{CP}_{\mathsf{link}}$. Their complexity, summarized in Table 1, results from the combined efficiency of the building blocks plus the observation that the matrices $\boldsymbol{W}_L, \boldsymbol{W}_R, \boldsymbol{W}_O$ are sparse and with a number of nonzero entries linear in the number of circuit wires.

CP-SNARKs for Arithmetic Circuits with Universal CRS. Both LegoAC1 and LegoAC2 have a circuit-specific CRS due to

the circuit-specific CRS of $\mathsf{CP}^{\mathsf{Ped}}_{\mathsf{lin}}$.[9] Aiming for a CP-SNARK for AC satisfiability with a linear-size CRS, we consider a different encoding of AC due to [17, 37]. Let C be an arithmetic circuit with N_A addition and N_M multiplication gates. Each gate has a left and right input and an output wire; each output wire can also be input to another gate. This means that C can be described by integers N_A, N_M, and the wiring information saying that the output wire of addition/multiplication i is the left/right input of addition/multiplication gate j. Having this in mind, satisfiability of C holds if there is a satisfying assignment to all the gates, i.e., $\boldsymbol{u}^A_L + \boldsymbol{u}^A_R = \boldsymbol{u}^A_O$ and $\boldsymbol{u}^M_L \circ \boldsymbol{u}^M_R = \boldsymbol{u}^M_O$, and if these assignments are consistent with C's wiring. This consistency is essentially a check that specific entries of the vector $(\boldsymbol{u}^A_L, \boldsymbol{u}^A_R, \boldsymbol{u}^A_O, \boldsymbol{u}^M_L, \boldsymbol{u}^M_R, \boldsymbol{u}^M_O)$ must be equal, which can be formalized with $R^{\mathsf{sfprm}}_{\phi}$ for an appropriate permutation ϕ.

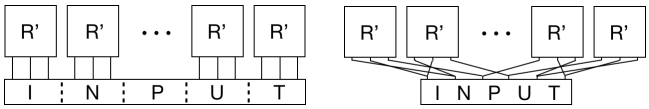
Using our composition theorem we obtain a CP-SNARK for AC satisfiability, dubbed LegoUAC, through the combination of R^{had} and R^{sfprm}. We propose to instantiate it with our $\mathsf{CP}_{\mathsf{had}}$ and $\mathsf{CP}_{\mathsf{sfprm}}$.[10] Both schemes admit a universal CRS that can be deterministically specialized (due to specializing $\mathsf{CP}_{\mathsf{sfprm}}$'s CRS to the circuit-dependent permutation ϕ). The complexity of LegoUAC is shown in Table 1 and stems from that of our $\mathsf{CP}_{\mathsf{had}}$ and $\mathsf{CP}_{\mathsf{sfprm}}$.

Parallel Checks on Joint Inputs. Consider a relation $R^{\mathsf{parjnt}}(u) := \bigwedge_{j=1}^{N} R'(u'_j)$ consisting of N parallel checks of the same relation R' on (partially) shared inputs. This relation has several use cases, e.g., proving knowledge of all the leaves of a Merkle tree of height N with respect to a public root, which can be seen as the parallel check of $2^N - 1$ hash verification relations (i.e., $R_H(x_1, x_2, y) := H(x_1, x_2) \stackrel{?}{=} y$) that share some of the inputs.

One way to deal with R^{parjnt} is by defining the arithmetic circuit that computes it (cf. Fig. 4b). The Hyrax system is particularly designed for parallel circuits [66]; they deal with non-parallel input by introducing a (non-parallel) redistribution layer (RDL) layer that *redistributes* the input and feeds it to the identical sub-circuits at the next level. Unfortunately an effect of using an RDL is that the verifier must pay an additional cost linear in the total width of the circuit. In several cases (including Merkle tree verification described earlier) this can result in a bottleneck in verification performances dominating its asymptotics.

We solve this issue by modeling R^{parjnt} as the simple conjunction of two relations: R^{par} for *fully parallel* checks of R' on *disjoint* inputs (cf. Fig. 4), and another relation that checks the consistency of the shared inputs across the parallel executions (that we can express with R^{sfprm} or R^{lin}). This way, we can build a CP-SNARK for R^{parjnt} (called LegoPar) through our composition result applied to our $\mathsf{CP}^{\mathsf{Ped}}_{\mathsf{lin}}$ and a CP-SNARK for R^{par} relations. For the latter, we use an adaptation of Hyrax using the polynomial commitment PolyCom of zk-vSQL. We call HyrPoly-Par this scheme invoked on circuits without RDL (i.e., it supports R^{par}), and HyrPoly-RDL the same scheme for circuits with an RDL (i.e., it supports R^{parjnt}).

We compare the efficiency of LegoPar and HyrPoly-RDL on R^{parjnt} relations. Let d and G be depth and width of the arithmetic circuit evaluating R'. Proving time and proof size have the same



(a) In R^{par}, R' inputs are disjoint **(b) In R^{parjnt}, R' inputs are joint**

Figure 4: Inputs structures for parallel relations.

complexity in both solutions; verifier time is $O(d(G + \log(NG)))$ in LegoPar and $O(d(G + \log(NG)) + |u| + NG)$ in HyrPoly-RDL. We note that due to the use of $\mathsf{CP}^{\mathsf{Ped}}_{\mathsf{lin}}$, the CRS of LegoPar becomes specific to the input wiring of R^{parjnt}, whereas in HyrPoly-RDL the CRS is just the commitment key. On the other hand, this one-time preprocessing allows the verifier to later check any number of proofs in shorter time.[11]

7 EXPERIMENTAL EVALUATION

We designed and implemented LegoSNARK,[12] a library for commit-and-prove SNARKs that includes a design framework for composable CP-SNARKs, and the implementation of a collection of proof gadgets: our $\mathsf{CP}_{\mathsf{link}}$ and $\mathsf{CP}^{\mathsf{Ped}}_{\mathsf{lin}}$, the Hadamard product CP-SNARK of [49], and the $\mathsf{CP}_{\mathsf{poly}}$ from [68][13]. LegoSNARK is written in C++; for polynomial operations and bilinear pairings we use the libraries underlying libsnark [3]. We executed our experiments on a virtual machine running Debian GNU/Linux with 8 Xeon Gold 6154 cores and 30 GB of RAM. We ran all tests single threaded. In our experiments, we tested the performance of some of our instantiations and compared to different baseline systems.

7.1 Commit-and-Prove SNARKs

We consider a generic application of proving commit-and-prove relations where commitments are created using the Pedersen scheme for vectors, i.e., proving $\exists(u, o, \omega)R(u, \omega) \wedge \mathsf{VerCommit}(\mathsf{ck}, c, u, o)$.

As baseline system, we use the Groth16 zkSNARK in libsnark on the libsnark gadget circuit for multi-scalar additions over a SNARK-friendly elliptic curve (to model the Pedersen computation). We call this CPGro16. We compare CPGro16 to a CP-SNARK, LegoGro16, obtained by applying our cc-SNARK-lifting compiler with our $\mathsf{CP}_{\mathsf{link}}$ scheme to the cc-variant of [39]. We measured the overhead of dealing with the commitment in both schemes (the R-dependent costs would be the same in both cases) at the increase of the committed vector's size (from 8 to 2048).[14] On the largest instance ($n = 2048$) LegoGro16's proving time is $5K\times$ (0.08 vs. 428 s) faster than CPGro16, at the price of a verification that is $1.2\times$ slower (4.1 vs 3.4 ms), and a slightly larger proof (191 vs. 127 Bytes). LegoGro16's CRS is also $7K\times$ shorter (130KB vs. 950MB).

In the case of LegoGro16, such overhead in proving time is essentially that of creating the additional element D of the proof that contains a commitment to to the data and to create a $\mathsf{CP}_{\mathsf{link}}$ proof to link D to the external commitment. The LegoGro16 proof is longer because of these two additional elements of $\mathbb{G}_1$. And for verification,

[9] Using our $\mathsf{CP}_{\mathsf{lin}}$ for PolyCom would give us an instantiation with a universal CRS, but unfortunately one of size $Q \cdot N$, that is quadratic in circuit size.

[10] Additions come for free by using a linearly homomorphic commitment.

[11] We do not see a way to run a similar preprocessing in HyrPoly. We evaluated the possibility to commit, in preprocessing, to the MLE of the RDL wiring so that the prover would compute this on behalf of the verifier and prove its correct evaluation using $\mathsf{CP}_{\mathsf{poly}}$. This idea however would require a commitment key quadratic in the circuit width, which is prohibitively large.

[12] A Github repository for the code is https://github.com/imdea-software/legosnark .

[13] For this we adapted to our library the code provided by the authors of [68].

[14] At $n = 4096$ CPGro16 ran out of memory.

the $\mathsf{CP}_{\mathsf{link}}$ verification must be executed. With respect to the CRS, in LegoGro16 we have the additional elements of the CRS needed to create D and the $\mathsf{CP}_{\mathsf{link}}$ CRS, that is essentially one vector of n elements of $\mathbb{G}_1$. In CPGro16, all the overhead in proving time and CRS is related to the size and degree of the QAP that models the computation of the Pedersen commitment. This was done by selecting an appropriate gadget in libsnark, which optimizes the task by selecting a suitable elliptic curve.

Table 3 shows our experimental results that compare the schemes LegoGro16 and CPGro16 with respect to the overhead for dealing with data committed using a Pedersen vector commitment. The experiments considered vectors of different length n.

	LegoGro16			CPGro16		
n	$\mathcal{KG}$ (ms)	$\mathcal{P}$ (ms)	crs (KB)	$\mathcal{KG}$ (s)	$\mathcal{P}$ (s)	crs (MB)
8	3.677	3.044	0.51	3.928	1.185	3.653
16	5.949	4.202	1.02	7.307	2.252	7.305
32	10.90	5.201	2.04	13.78	4.461	14.61
64	19.37	8.979	4.08	26.04	8.685	29.22
128	32.49	15.58	8.16	50.69	16.50	58.44
256	57.76	19.50	16.32	102.8	33.02	116.9
512	117.8	30.84	32.64	292.0	65.42	233.7
1024	241.2	55.35	65.28	876.3	133.3	467.5
2048	466.6	84.09	130.6	1011	428.7	935.0
$\lvert\pi\rvert$ (B)	191.13			127.38		
$\mathcal{V}$ (ms)	4.129			3.4		

Table 3: Performance comparison of LegoGro16 **and** CPGro16**. Numbers for the two schemes are in different units. Those for** CPGro16 **are three orders of magnitude larger.**

7.2 Matrix Multiplication

We evaluate our LegoMM scheme for matrix multiplication against a solution based on Groth16 [39]. Here, we use the version of our CP-SNARK where the output matrix is given in clear as part of the statement to be proven (i.e. $\mathsf{CP}_{\mathsf{mmp}}$). Our scheme has a faster prover and smaller crs using an asymptotically more efficient verifier with a longer, but still succinct, proof. Our experiments confirm the theoretical costs of these schemes.

We evaluate times both proving and verification time when delegating a square matrix multiplication with size $N = n \times n$ field elements, ranging from $n = 16$ to $n = 256$. We observe our scheme noticeably improves on proving time as our prover runs in linear time in the number of elements in the matrix (n^2), whereas Groth's runs in quasicubic time in n. Even if our verifier is slower for smaller matrices, the $O(n^2)$ work in our scheme involves only *field operations* whereas in Groth16 one needs to do a $O(n^2)$-wide multiexponentiation. On the largest instance we measured (square matrices with $n = 128$ rows and columns), our proving time is roughly 1300× faster (109 seconds vs. 84 milliseconds) and verification time is 1.8× faster (51 vs. 28 milliseconds). This is a tradeoff between the running time and the proof length: only 3 group elements in Groth16 vs $O(\log n)$ in our scheme (96 bytes vs. 32 kilobytes).

Table 4 shows concrete performance measurements of both schemes, showing a clear proving time improvement in our scheme.

	Time		Space	
	$\mathcal{P}$	$\mathcal{V}$	crs	π
LegoMM	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(\log n)$
Groth16	$O(n^3 \log n)$	$O(n^2)$	$O(n^3)$	$O(1)$

	LegoMM			Groth16	
n	$\mathcal{P}$ (ms)	$\mathcal{V}$ (ms)	$\lvert\pi\rvert$ KB	$\mathcal{P}$ (s)	$\mathcal{V}$ (ms)
16	35.36	22.74	21	0.181	4.312
32	46.26	23.19	24	1.379	6.060
64	55.78	24.00	28	11.61	12.60
128	83.78	28.03	32	109.3	50.99
256	149.7	40.01	36		

Table 4: Comparing LegoMM **and Groth16 for** $n \times n$ **matrices**

7.3 LegoAC1 for Arithmetic Circuits

We tested our LegoAC1 scheme (see Section 6) for arithmetic circuits and compared it to Groth16 as a baseline system. We considered two benchmark applications:

(a) proving knowledge of a *SHA256 pre-image* on 512-bit inputs; for this we used the existing circuit gadgets implemented in libsnark (for Groth16), and in Bulletproofs [2] (for LegoAC1).

(b) *matrix factoring*, i.e., proving knowledge of two $n \times n$ matrices A, B whose product is a public matrix C; for this we designed suitable constraints systems, considering 32-bit integers entries and a varying $n = 16, 32, 64, 128$.

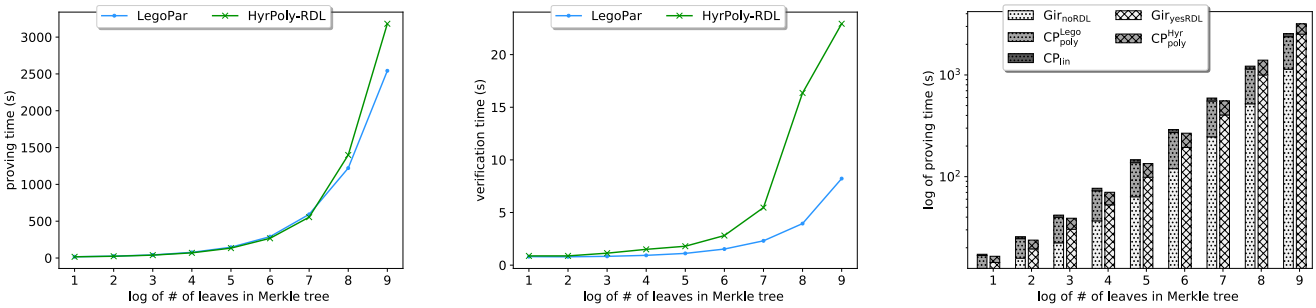
Overall, our experiments show that LegoAC1 performs slightly worse than Groth16. For example, for SHA256 proving time is 1.2× slower (0.7 vs. 0.9 s); verification is up to 2× slower (0.9 vs. 1.8 ms) and improves with larger inputs; our key generation is about $5-6\times$ slower. Proof size is constant: 350B in LegoAC1 and 127B in Groth16. Noteworthy that most of LegoAC1 key generation time (about 70%) is taken by the corresponding algorithm for $\mathsf{CP}^{\mathsf{Ped}}_{\mathsf{lin}}$;

	LegoAC1			Groth16		
n	$\mathcal{KG}$ (s)	$\mathcal{P}$ (s)	$\mathcal{V}$ (ms)	$\mathcal{KG}$ (s)	$\mathcal{P}$ (s)	$\mathcal{V}$ (ms)
16	1.105	0.278	3.097	0.210	0.150	1.662
32	7.569	1.680	4.697	1.227	0.957	3.696
64	52.86	11.90	10.73	8.848	7.177	9.686
128	419.8	89.70	35.71	69.21	58.60	34.83
$\lvert\pi\rvert$ (B)	350.25			127.38		

Table 5: Performance of LegoAC1 **comparing to Groth16**

this is mainly due to an unoptimized technique for dealing with sparse matrices like the ones that encode the linear constraints $\boldsymbol{W}_L, \boldsymbol{W}_R, \boldsymbol{W}_O$, and we expect this to be improved in the future.

In a way this result is not surprising: Groth16 is an extremely optimized and well explored scheme, whereas for LegoAC1 we believe that more optimizations could be explored (in a similar way as Groth16 optimized Pinocchio). More remarkably, LegoAC1 *has a built-in commit-and-prove capability*, which means its proofs are done with respect to matrices that committed in a Pedersen



(a) Proving time comparison for LegoPar **and** HyrPoly-RDL.

(b) Verification time comparison for LegoPar **and** HyrPoly-RDL.

(c) $\mathcal{P}$ time (component-wise) for LegoPar **(left bars) and** HyrPoly-RDL **(right).**

Figure 5: Performance comparison of systems for parallel relations. Lower on the y axis is better (in (c), axis y is log-scale). We remind the reader that LegoPar = HyrPoly-Par + $\mathsf{CP}_{\mathsf{lin}}$ = ($\mathsf{Gir}_{\mathsf{noRDL}}$ + $\mathsf{CP}^{\mathsf{Lego}}_{\mathsf{poly}}$) + $\mathsf{CP}_{\mathsf{lin}}$ **and** HyrPoly-RDL = $\mathsf{Gir}_{\mathsf{yesRDL}}$ + $\mathsf{CP}^{\mathsf{Hyr}}_{\mathsf{poly}}$.

commitment (in a canonical vectorized form). This property is not present in Groth16, and can be useful in several applications.

For example, in the matrix factoring case, LegoAC1 works with commitments to the three matrices that could be reused. This is a powerful feature as we could prove a statement like "$B = A^{2^k}$ for a committed matrix A" by doing k proofs, one for each squaring step (i.e., to show that $B_i = B_{i-1}^2$); this can be done by reusing the same CRS for one matrix factoring relation. In contrast, proving $B = A^{2^k}$ directly with Groth16 would require a very large CRS and a memory intensive prover that would not scale for large k and n.

We give the experimental results that compare our LegoAC1 commit-and-prove zkSNARK against the Groth16 scheme, in the SHA256 and matrix factoring applications explained above. For SHA256, Groth16 needs $1.9s$ for key generation of a CRS of $5.1MB$, $0.7s$ for proving and $0.9ms$ for verification; LegoAC1 needs $7.9s$ for key generation of a CRS of $6.2MB$, $0.9s$ for proving and $1.8ms$ for verification. For matrix factoring, we used $n \times n$ matrices of 32-bit integers with $n \in \{16, 32, 64, 128\}$. Detailed timings are in Table 5.

7.4 Parallel Checks on Joint Inputs

We compare performances of our LegoPar system with a baseline system, i.e. HyrPoly-RDL. Our choice of a Hyrax-based system for comparison is due to its optimization for parallel computations, and thus enhancing the latter implies refinements in all computations where Hyrax applies today.

Recall that LegoPar consists of our $\mathsf{CP}^{\mathsf{Ped}}_{\mathsf{lin}}$ and HyrPoly-Par. To evaluate HyrPoly-Par and HyrPoly-RDL we executed separately the part concerning PolyCom and $\mathsf{CP}_{\mathsf{poly}}$, and the one that includes the ZKGir^{++} core. To benchmark the latter, we used the original Python code (appropriately modified) from the Hyrax project [1] (run using the JIT-compiling interpreter PyPy [4]).

We benchmarked LegoPar and HyrPoly-RDL on a highly parallel computation, that is proving knowledge of an assignment to the leaves of a Merkle tree [51] (cf. Section 6 to see how it can be expressed using R^{parjnt}). We used SHA256 for the hash and a varying number of leaves (from 2 to 2^9). For this computation we generated two circuits using the Hyrax tool: one fully parallel to be fed to HyrPoly-Par and one with the RDL for HyrPoly-RDL. Recall that in LegoPar the RDL is checked using $\mathsf{CP}^{\mathsf{Ped}}_{\mathsf{lin}}$. We finally note that the two largest inputs in our evaluation required extending the available RAM from 30 to 75GB for both schemes.

Results. Figure 5 compares the costs (proving and verification time) in the two schemes for repeated computation. Overall LegoPar is faster than HyrPoly-RDL, both in proving and verification time. On our largest input, proving in LegoPar is 1.25× faster; verifying is more than 2.5× faster. Verification is expected to become faster due to the asymptotic difference in the verification time.

- *Proving time*: On larger inputs LegoPar has a faster (up to 1.25×) proving time (Figs. 5a). In both schemes most of the computation is due to ZKGir^{++}: approximately 50% for LegoPar and 75% for HyrPoly-RDL. The higher time of ZKGir^{++} in HyrPoly-RDL is explained by the additional round for the RDL. On the other hand, LegoPar spends twice as much time for the proving step of $\mathsf{CP}_{\mathsf{poly}}$. This is because it evaluates a polynomial with twice as many terms, in turn requiring roughly twice the number of exponentiations. (This is due to the RDL output u_2, on which LegoPar operates, being twice as long as the RDL input u_1 (also the "bottom-layer" input), on which $\mathsf{CP}_{\mathsf{poly}}$ runs in HyrPoly-RDL).
- *Verification time*: On larger inputs LegoPar has a shorter (up to 2.5×) verification time (Fig. 5b). This speedup is due to increase with larger inputs, as the verifier in HyrPoly-RDL has to perform an additional verification step for the RDL in ZKGir^{++} (requiring a number of field operations roughly linear in the width of the circuit). On the other hand LegoPar performs the same step through a constant number of pairings (two) in $\mathsf{CP}_{\mathsf{veq}}$. In both schemes ZKGir^{++} dominates running time (more than 99.5%).

Discussion. Partly, the different performances we observed are due to specific features of the circuit chosen for benchmarks (we chose Merkle tree verification, due to its relevance in practice). In a circuit for parallel computation, at least two features, both related to the RDL, can have impact: *(i)* how "large" the output u_2 of the RDL is w.r.t. its input u_1; *(ii)* how "complex" the RDL is. A higher ratio $|u_2|/|u_1|$ will determine the difference in running time for the $\mathsf{CP}_{\mathsf{poly}}$.Prove component. In our circuit of choice the ratio was 2.

8 CONCLUSIONS

We have described LegoSNARK, a framework for commit-and-prove zkSNARKs that comprises definitions, a general composition result, and a "lifting" construction. The LegoSNARK tools are useful as

they enable designing zkSNARKs in a modular way (due to the framework of definitions and the composition theorem) and they allow to efficiently add commit-and-prove capabilities to a variety of existing schemes thus made interoperable. Furthermore we have proposed efficient proof gadgets for specialized relations and shown how to combine them into succinct proof systems for more complex relations. We have described instantiations of these new proof systems and evaluated them against prior work. The results show they have competitive performances. Specifically they show slightly worse (but still acceptable) performances in some applications (general arithmetic circuits) and significant improvements in others (commit-ahead-of-time systems, parallel computations).

A limitation of our current instantiations is their reliance on pairing-based systems with a trusted setup. Interestingly in some cases this is only needed to generate the commitment key of PolyCom. We believe this is doable by a large-scale MPC ceremony similar to the powers-of-tau round 1 of [18] since the CRS includes only monomials in the exponent. It is future work to explore this direction. Nonetheless we note that this limitation is not inherent. The basic results of the framework (i.e., Section 3) are general enough to be instantiated in the future with schemes without trust assumptions.

Finally, another future work direction is investigating new and more efficient proof gadgets CP-SNARKs for specialized relations and test them in specific applications.

ACKNOWLEDGEMENTS

Research leading to these results was supported by the Spanish Government under projects Datamantium (ref. RTC-2016-4930-7), SCUM (ref. RTI2018-102043-B-I00), and ERC2018-092822, by the Madrid Regional Government under project BLOQUES (ref. S2018/TCS-4339), and by Protocol Labs.

The project that gave rise to these results received the support of a fellowship from "la Caixa" Foundation (ID 100010434). The fellowship code is LCF/BQ/ES18/11670018.

REFERENCES

[1] Hyrax. https://github.com/hyraxZK.
[2] libsecp256k1. https://github.com/apoelstra/secp256k1-mw/tree/bulletproofs.
[3] libsnark. https://github.com/scipr-lab/libsnark.
[4] PyPy. https://pypy.org.
[5] Shashank Agrawal, Chaya Ganesh, and Payman Mohassel. 2018. Non-Interactive Zero-Knowledge Proofs for Composite Statements. In *CRYPTO 2018, Part III (LNCS)*, Hovav Shacham and Alexandra Boldyreva (Eds.), Vol. 10993. Springer, Heidelberg, 643–673. https://doi.org/10.1007/978-3-319-96878-0_22
[6] Kurt M. Alonso and Jordi Herrera Joancomartí. 2018. Monero - Privacy in the Blockchain. Cryptology ePrint Archive, Report 2018/535. https://eprint.iacr.org/2018/535.
[7] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. 2017. Ligero: Lightweight Sublinear Arguments Without a Trusted Setup. In *ACM CCS 17*, Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu (Eds.). ACM Press, 2087–2104. https://doi.org/10.1145/3133956.3134104
[8] Michael Backes, Manuel Barbosa, Dario Fiore, and Raphael M. Reischuk. 2015. ADSNARK: Nearly Practical and Privacy-Preserving Proofs on Authenticated Data. In *2015 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 271–286. https://doi.org/10.1109/SP.2015.24
[9] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. 2018. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Report 2018/046. https://eprint.iacr.org/2018/046.
[10] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. 2014. Zerocash: Decentralized Anonymous Payments from Bitcoin. In *2014 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 459–474. https://doi.org/10.1109/SP.2014.36
[11] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer, and Madars Virza. 2013. SNARKs for C: Verifying Program Executions Succinctly and in Zero Knowledge. In *CRYPTO 2013, Part II (LNCS)*, Ran Canetti and Juan A. Garay (Eds.), Vol. 8043. Springer, Heidelberg, 90–108. https://doi.org/10.1007/978-3-642-40084-1_6
[12] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. 2016. Interactive Oracle Proofs. In *TCC 2016-B, Part II (LNCS)*, Martin Hirt and Adam D. Smith (Eds.), Vol. 9986. Springer, Heidelberg, 31–60. https://doi.org/10.1007/978-3-662-53644-5_2
[13] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. 2014. Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture. In *USENIX Security*. 781–796.
[14] Nir Bitansky, Ran Canetti, Alessandro Chiesa, Shafi Goldwasser, Huijia Lin, Aviad Rubinstein, and Eran Tromer. 2017. The Hunting of the SNARK. *Journal of Cryptology* 30, 4 (Oct. 2017), 989–1066.
[15] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. 2012. From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. In *ITCS 2012*, Shafi Goldwasser (Ed.). ACM, 326–349. https://doi.org/10.1145/2090236.2090263
[16] Jonathan Bootle, Andrea Cerulli, Pyrros Chaidos, Jens Groth, and Christophe Petit. 2016. Efficient Zero-Knowledge Arguments for Arithmetic Circuits in the Discrete Log Setting. In *EUROCRYPT 2016, Part II (LNCS)*, Marc Fischlin and Jean-Sébastien Coron (Eds.), Vol. 9666. Springer, Heidelberg, 327–357. https://doi.org/10.1007/978-3-662-49896-5_12
[17] Jonathan Bootle, Andrea Cerulli, Essam Ghadafi, Jens Groth, Mohammad Hajiabadi, and Sune K. Jakobsen. 2017. Linear-Time Zero-Knowledge Proofs for Arithmetic Circuit Satisfiability. In *ASIACRYPT 2017, Part III (LNCS)*, Tsuyoshi Takagi and Thomas Peyrin (Eds.), Vol. 10626. Springer, Heidelberg, 336–365. https://doi.org/10.1007/978-3-319-70700-6_12
[18] Sean Bowe, Ariel Gabizon, and Ian Miers. 2017. Scalable Multi-party Computation for zk-SNARK Parameters in the Random Beacon Model. Cryptology ePrint Archive, Report 2017/1050. https://eprint.iacr.org/2017/1050.
[19] Benjamin Braun, Ariel J. Feldman, Zuocheng Ren, Srinath Setty, Andrew J. Blumberg, and Michael Walfish. 2013. Verifying computations with state. In *Proc. of the ACM SOSP*.
[20] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. 2017. *Bulletproofs: Efficient range proofs for confidential transactions*. Technical Report. Cryptology ePrint Archive, Report 2017/1066, 2017. https://eprint. iacr. org/2017/1066.
[21] Matteo Campanelli, Dario Fiore, and Anaïs Querol. 2019. LegoSNARK: Modular Design and Composition of Succinct Zero-Knowledge Proofs. Cryptology ePrint Archive, Report 2019/142. http://eprint.iacr.org/.
[22] Ran Canetti, Yehuda Lindell, Rafail Ostrovsky, and Amit Sahai. 2002. Universally composable two-party and multi-party secure computation. In *34th ACM STOC*. ACM Press, 494–503. https://doi.org/10.1145/509907.509980
[23] Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, and Greg Zaverucha. 2017. Post-Quantum Zero-Knowledge and Signatures from Symmetric-Key Primitives. In *ACM CCS 17*, Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu (Eds.). ACM Press, 1825–1842. https://doi.org/10.1145/3133956.3133997
[24] Melissa Chase, Chaya Ganesh, and Payman Mohassel. 2016. Efficient Zero-Knowledge Proof of Algebraic and Non-Algebraic Statements with Applications to Privacy Preserving Credentials. In *CRYPTO 2016, Part III (LNCS)*, Matthew Robshaw and Jonathan Katz (Eds.), Vol. 9816. Springer, Heidelberg, 499–530. https://doi.org/10.1007/978-3-662-53015-3_18
[25] Graham Cormode, Michael Mitzenmacher, and Justin Thaler. 2012. Practical verified computation with streaming interactive proofs. In *ITCS 2012*, Shafi Goldwasser (Ed.). ACM, 90–112. https://doi.org/10.1145/2090236.2090245
[26] Craig Costello, Cédric Fournet, Jon Howell, Markulf Kohlweiss, Benjamin Kreuter, Michael Naehrig, Bryan Parno, and Samee Zahur. 2015. Geppetto: Versatile Verifiable Computation. In *2015 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 253–270. https://doi.org/10.1109/SP.2015.23
[27] Alex Escala, Gottfried Herold, Eike Kiltz, Carla Ràfols, and Jorge Villar. 2013. An Algebraic Framework for Diffie-Hellman Assumptions. In *CRYPTO 2013, Part II (LNCS)*, Ran Canetti and Juan A. Garay (Eds.), Vol. 8043. Springer, Heidelberg, 129–147. https://doi.org/10.1007/978-3-642-40084-1_8
[28] Prastudy Fauzi, Helger Lipmaa, Janno Siim, and Michal Zajac. 2017. An Efficient Pairing-Based Shuffle Argument. In *ASIACRYPT 2017, Part II (LNCS)*, Tsuyoshi Takagi and Thomas Peyrin (Eds.), Vol. 10625. Springer, Heidelberg, 97–127. https://doi.org/10.1007/978-3-319-70697-9_4
[29] Dario Fiore, Cédric Fournet, Esha Ghosh, Markulf Kohlweiss, Olga Ohrimenko, and Bryan Parno. 2016. Hash First, Argue Later: Adaptive Verifiable Computations on Outsourced Data. In *ACM CCS 16*, Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi (Eds.). ACM Press, 1304–1316. https://doi.org/10.1145/2976749.2978368
[30] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. 2018. The Algebraic Group Model and its Applications. In *CRYPTO 2018, Part II (LNCS)*, Hovav Shacham and Alexandra Boldyreva (Eds.), Vol. 10992. Springer, Heidelberg, 33–62. https://doi.org/10.1007/978-3-319-96881-0_2

[31] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. 2013. Quadratic Span Programs and Succinct NIZKs without PCPs. In *EUROCRYPT 2013 (LNCS)*, Thomas Johansson and Phong Q. Nguyen (Eds.), Vol. 7881. Springer, Heidelberg, 626–645. https://doi.org/10.1007/978-3-642-38348-9_37

[32] Craig Gentry and Daniel Wichs. 2011. Separating succinct non-interactive arguments from all falsifiable assumptions. In *43rd ACM STOC*, Lance Fortnow and Salil P. Vadhan (Eds.). ACM Press, 99–108. https://doi.org/10.1145/1993636.1993651

[33] Irene Giacomelli, Jesper Madsen, and Claudio Orlandi. 2016. ZKBoo: Faster Zero-Knowledge for Boolean Circuits. In *25th USENIX Security Symposium (USENIX Security 16)*. USENIX Association, Austin, TX, 1069–1083.

[34] Oded Goldreich, Silvio Micali, and Avi Wigderson. 1987. How to Play any Mental Game or A Completeness Theorem for Protocols with Honest Majority. In *19th ACM STOC*, Alfred Aho (Ed.). ACM Press, 218–229. https://doi.org/10.1145/28395.28420

[35] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. 2008. Delegating computation: interactive proofs for muggles. In *40th ACM STOC*, Richard E. Ladner and Cynthia Dwork (Eds.). ACM Press, 113–122. https://doi.org/10.1145/1374376.1374396

[36] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. 1989. The Knowledge Complexity of Interactive Proof Systems. *SIAM J. Comput.* 18, 1 (1989), 186–208.

[37] Jens Groth. 2009. Linear Algebra with Sub-linear Zero-Knowledge Arguments. In *CRYPTO 2009 (LNCS)*, Shai Halevi (Ed.), Vol. 5677. Springer, Heidelberg, 192–208. https://doi.org/10.1007/978-3-642-03356-8_12

[38] Jens Groth. 2010. Short Pairing-Based Non-interactive Zero-Knowledge Arguments. In *ASIACRYPT 2010 (LNCS)*, Masayuki Abe (Ed.), Vol. 6477. Springer, Heidelberg, 321–340. https://doi.org/10.1007/978-3-642-17373-8_19

[39] Jens Groth. 2016. On the Size of Pairing-Based Non-interactive Arguments. In *EUROCRYPT 2016, Part II (LNCS)*, Marc Fischlin and Jean-Sébastien Coron (Eds.), Vol. 9666. Springer, Heidelberg, 305–326. https://doi.org/10.1007/978-3-662-49896-5_11

[40] Jens Groth, Markulf Kohlweiss, Mary Maller, Sarah Meiklejohn, and Ian Miers. 2018. Updatable and Universal Common Reference Strings with Applications to zk-SNARKs. In *CRYPTO 2018, Part III (LNCS)*, Hovav Shacham and Alexandra Boldyreva (Eds.), Vol. 10993. Springer, Heidelberg, 698–728. https://doi.org/10.1007/978-3-319-96878-0_24

[41] Daniel Günther, Ágnes Kiss, and Thomas Schneider. 2017. More Efficient Universal Circuit Constructions. In *ASIACRYPT 2017, Part II (LNCS)*, Tsuyoshi Takagi and Thomas Peyrin (Eds.), Vol. 10625. Springer, Heidelberg, 443–470. https://doi.org/10.1007/978-3-319-70697-9_16

[42] Yuval Ishai, Eyal Kushilevitz, and Rafail Ostrovsky. 2007. Efficient Arguments Without Short PCPs. In *Proceedings of the Twenty-Second Annual IEEE Conference on Computational Complexity (CCC '07)*. IEEE Computer Society, Washington, DC, USA, 278–291.

[43] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. 2007. Zero-knowledge from secure multiparty computation. In *39th ACM STOC*, David S. Johnson and Uriel Feige (Eds.). ACM Press, 21–30. https://doi.org/10.1145/1250790.1250794

[44] J. Kilian. 1989. Uses of Randomness in Algorithms and Protocols. PhD Thesis. Massachusetts Institute of Technology.

[45] Joe Kilian. 1992. A Note on Efficient Zero-Knowledge Proofs and Arguments (Extended Abstract). In *24th ACM STOC*. ACM Press, 723–732. https://doi.org/10.1145/129712.129782

[46] Eike Kiltz and Hoeteck Wee. 2015. Quasi-Adaptive NIZK for Linear Subspaces Revisited. In *EUROCRYPT 2015, Part II (LNCS)*, Elisabeth Oswald and Marc Fischlin (Eds.), Vol. 9057. Springer, Heidelberg, 101–128. https://doi.org/10.1007/978-3-662-46803-6_4

[47] Ahmed E. Kosba, Dimitrios Papadopoulos, Charalampos Papamanthou, Mahmoud F. Sayed, Elaine Shi, and Nikos Triandopoulos. 2014. TRUESET: Faster Verifiable Set Computations. In *USENIX Security*. 765–780.

[48] Helger Lipmaa. 2012. Progression-Free Sets and Sublinear Pairing-Based Non-Interactive Zero-Knowledge Arguments. In *TCC 2012 (LNCS)*, Ronald Cramer (Ed.), Vol. 7194. Springer, Heidelberg, 169–189. https://doi.org/10.1007/978-3-642-28914-9_10

[49] Helger Lipmaa. 2016. Prover-Efficient Commit-and-Prove Zero-Knowledge SNARKs. In *AFRICACRYPT 16 (LNCS)*, David Pointcheval, Abderrahmane Nitaj, and Tajjeeddine Rachidi (Eds.), Vol. 9646. Springer, Heidelberg, 185–206. https://doi.org/10.1007/978-3-319-31517-1_10

[50] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. 1992. Algebraic Methods for Interactive Proof Systems. *J. ACM* 39, 4 (Oct. 1992), 859–868.

[51] Ralph C. Merkle. 1988. A Digital Signature Based on a Conventional Encryption Function. In *CRYPTO'87 (LNCS)*, Carl Pomerance (Ed.), Vol. 293. Springer, Heidelberg, 369–378. https://doi.org/10.1007/3-540-48184-2_32

[52] Silvio Micali. 1994. CS Proofs (Extended Abstracts). In *35th FOCS*. IEEE Computer Society Press, 436–453. https://doi.org/10.1109/SFCS.1994.365746

[53] Silvio Micali. 2000. Computationally Sound Proofs. *SIAM J. Comput.* 30, 4 (2000), 1253–1298. https://doi.org/10.1137/S0097539795284959

[54] Moni Naor and Moti Yung. 1990. Public-key Cryptosystems Provably Secure against Chosen Ciphertext Attacks. In *22nd ACM STOC*. ACM Press, 427–437. https://doi.org/10.1145/100216.100273

[55] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. 2013. Pinocchio: Nearly Practical Verifiable Computation. In *2013 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 238–252. https://doi.org/10.1109/SP.2013.47

[56] Torben P. Pedersen. 1992. Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing. In *CRYPTO'91 (LNCS)*, Joan Feigenbaum (Ed.), Vol. 576. Springer, Heidelberg, 129–140. https://doi.org/10.1007/3-540-46766-1_9

[57] Omer Reingold, Guy N. Rothblum, and Ron D. Rothblum. 2016. Constant-round interactive proofs for delegating computation. In *48th ACM STOC*, Daniel Wichs and Yishay Mansour (Eds.). ACM Press, 49–62. https://doi.org/10.1145/2897518.2897652

[58] Guy Rothblum. 2009. Delegating computation reliably: paradigms and constructions. PhD thesis.

[59] Claus-Peter Schnorr. 1991. Efficient Signature Generation by Smart Cards. *Journal of Cryptology* 4, 3 (1991), 161–174.

[60] Justin Thaler. 2013. Time-Optimal Interactive Proofs for Circuit Evaluation. In *CRYPTO 2013, Part II (LNCS)*, Ran Canetti and Juan A. Garay (Eds.), Vol. 8043. Springer, Heidelberg, 71–89. https://doi.org/10.1007/978-3-642-40084-1_5

[61] Leslie G. Valiant. 1976. Universal Circuits (Preliminary Report). In *STOC*. ACM, 196–203.

[62] Meilof Veeningen. 2017. Pinocchio-Based Adaptive zk-SNARKs and Secure/Correct Adaptive Function Evaluation. In *AFRICACRYPT 17 (LNCS)*, Marc Joye and Abderrahmane Nitaj (Eds.), Vol. 10239. Springer, Heidelberg, 21–39.

[63] Riad S. Wahby, Ye Ji, Andrew J. Blumberg, Abhi Shelat, Justin Thaler, Michael Walfish, and Thomas Wies. 2017. Full Accounting for Verifiable Outsourcing. In *ACM CCS 17*, Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu (Eds.). ACM Press, 2071–2086. https://doi.org/10.1145/3133956.3133984

[64] Riad S. Wahby, Srinath T. V. Setty, Zuocheng Ren, Andrew J. Blumberg, and Michael Walfish. 2015. Efficient RAM and control flow in verifiable outsourced computation. In *NDSS 2015*. The Internet Society.

[65] Riad S. Wahby, Ioanna Tzialla, abhi shelat, Justin Thaler, and Michael Walfish. 2017. Doubly-efficient zkSNARKs without trusted setup. Cryptology ePrint Archive, Report 2017/1132. https://eprint.iacr.org/2017/1132.

[66] Riad S. Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. 2018. Doubly-Efficient zkSNARKs Without Trusted Setup. In *2018 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 926–943. https://doi.org/10.1109/SP.2018.00060

[67] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. 2017. vSQL: Verifying Arbitrary SQL Queries over Dynamic Outsourced Databases. In *2017 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 863–880. https://doi.org/10.1109/SP.2017.43

[68] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. 2017. A Zero-Knowledge Version of vSQL. Cryptology ePrint Archive, Report 2017/1146. https://eprint.iacr.org/2017/1146.